



Bangladesh University of Engineering and Technology

BUET_GreyMatter

Amartya Sarker, Shahtaz Rahman Labib, Raad Sharar

2025-12-19

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

template.cpp34 lines

```
#include<bits/stdc++.h>

using namespace std;

#define int long long
typedef long long ll;
typedef vector<int> vi;
typedef vector<bool> vb;
typedef vector<vi> vvi;

#define All(a) a.begin(),a.end()
#define FOR(i,a,b) for(int i=a;i<b;i++)
template<typename T> void get_vector(T&a){for(auto&e:a)cin>>e;}
template<typename T> void put_vector(T a){for(auto e:a)cout<<e<<" ";cout<<endl;}

const int inf = INT_MAX;
const int INF = 1e18;

void solve(){

}

signed main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int T = 1;
    cin>>T;
    while(T--){
        solve();
    }
    return 0;
}
```

1 .bashrc3 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
-fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps =<
```

1 .vimrc6 lines

```
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul
sy on | im jk <esc> | im kj <esc> | no ; :
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \|
\| md5sum \| cut -c-6
```

hash.sh3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

troubleshoot.txt52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
```

What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Mathematics (2)

Data structures (3)

OrderStatisticTree.h782797, 16 lines

```
Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type.
Time: O(log N)

#include <bits/extc++.h>
using namespace __gnu_pbds;

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

void example() {
    Tree<int> t, t2; t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.hd77092, 7 lines

```
Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
    const uint64_t C = 11(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int, chash> h({}, {}, {}, {}, {1<16});
```

SegmentTree.h405109, 167 lines

```
Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).
<bits/stdc++.h>

struct Tree {
    typedef int T;
    static constexpr T unit = INT_MIN;
    T f(T a, T b) { return max(a, b); } // (any associative fn)
    vector<T> s; int n;
    Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
    void update(int pos, T val) {
        for (s[pos += n] = val; pos /= 2;)
            s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
    }
    T query(int b, int e) { // query [b, e)
        T ra = unit, rb = unit;
        for (b += n, e += n; b < e; b /= 2, e /= 2) {
            if (b % 2) ra = f(ra, s[b++]);
            if (e % 2) rb = f(s[--e], rb);
        }
        return f(ra, rb);
    }
};
```

```

    }
};

// #####

using namespace std;
#define int long long

const int inf = INT_MAX;
const int INF = 1e18;

class SegmentTree{
    struct item{
        int x;
    };

    item NUTRAL_ITEM = {0}; // merge(a , NUTRAL_ITEM) = a
    vector<item> values;
    int n;

    item merge(item a, item b){
        return {a.x+b.x};
    }

    item single(int v){
        return {v};
    }

    void build(vector<int> &a,int at, int b, int e){
        if(b == e){
            values[at] = single(a[b]);
            return;
        }

        int mid = (b+e)>>1;
        build(a,at*2,b,mid);
        build(a,at*2+1,mid+1,e);
        values[at] = merge(values[at*2],values[at*2+1]);
    }
    //update(1,0,n-1,i,v)
    void update(int at,int b, int e, int i,int v){
        if (b > i || e < i) return;
        if (b == e && b == i) {
            values[at] = single(v);
            return;
        }

        int mid = (b+e)>>1;
        update(at*2,b,mid,i,v);
        update(at*2+1,mid+1,e,i,v);
        values[at] = merge(values[at*2],values[at*2+1]);
    }
    //query(1,0,n-1,l,r) *** (l,r are 0 indeded)
    item query(int at, int b, int e, int l, int r) {
        if (b > r || e < l) {
            return NUTRAL_ITEM;
        }
        if (l <= b && e <= r) {
            return values[at];
        }

        int mid = (b + e) >> 1;
        item left = query(at * 2, b, mid, l, r);
        item right = query(at * 2 + 1, mid + 1, e, l, r);
        return merge(left, right);
    }

public:
    SegmentTree(vector<int>&a){

```

```

        n = a.size();
        values.resize(n*4);
        build(a,1,0,n-1);
    }

    item query(int l,int r){
        return query(1,0,n-1,l,r);
    }

    void update(int i, int v){
        update(1,0,n-1,i,v);
    }
};

using namespace std;

const int N = 1e5 + 9;

struct PST {
#define lc t[cur].l
#define rc t[cur].r
    struct node {
        int l = 0, r = 0, val = 0;
    } t[20 * N];
    int T = 0;
    int build(int b, int e) {
        int cur = ++T;
        if(b == e) return cur;
        int mid = b + e >> 1;
        lc = build(b, mid);
        rc = build(mid + 1, e);
        t[cur].val = t[lc].val + t[rc].val;
        return cur;
    }

    int upd(int pre, int b, int e, int i, int v) {
        int cur = ++T;
        t[cur] = t[pre];
        if(b == e) {
            t[cur].val += v;
            return cur;
        }
        int mid = b + e >> 1;
        if(i <= mid) {
            rc = t[pre].r;
            lc = upd(t[pre].l, b, mid, i, v);
        } else {
            lc = t[pre].l;
            rc = upd(t[pre].r, mid + 1, e, i, v);
        }
        t[cur].val = t[lc].val + t[rc].val;
        return cur;
    }

    int query(int pre, int cur, int b, int e, int k) {
        if(b == e) return b;
        int cnt = t[lc].val - t[t[pre].l].val;
        int mid = b + e >> 1;
        if(cnt >= k) return query(t[pre].l, lc, b, mid, k);
        else return query(t[pre].r, rc, mid + 1, e, k - cnt);
    }
} t;

//the code returns k-th number in a range l to r if the range
//were sorted
int V[N], root[N], a[N];
int32_t main() {
    map<int, int> mp;
    int n, q;
    cin >> n >> q;
    for(int i = 1; i <= n; i++) cin >> a[i], mp[a[i]];
    int c = 0;
    for(auto x : mp) mp[x.first] = ++c, V[c] = x.first;
    root[0] = t.build(1, n);

```

```

    for(int i = 1; i <= n; i++) {
        root[i] = t.upd(root[i - 1], 1, n, mp[a[i]], 1);
    }
    while(q--) {
        int l, r, k;
        cin >> l >> r >> k;
        cout << V[t.query(root[l - 1], root[r], 1, n, k)] << '\n';
    }
    return 0;
}

```

LazySegmentTree.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

7a1959, 112 lines

```

class SegmentTree{
    vector<int> values;
    vector<pair<int,bool>> lazy; // {value, 1 if set 0 if update}

    int n;
    //function to combine two segment
    int merge(int a, int b){
        return a+b;
    }

    void build(vector<int> &a,int at, int b, int e){
        if(b == e){
            values[at] = a[b];
            return;
        }

        int mid = (b+e)>>1;
        build(a,at*2,b,mid);
        build(a,at*2+1,mid+1,e);
        values[at] = merge(values[at*2],values[at*2+1]);
    }

    // pushing pending updates to children. lazy propation
    void push(int at, int b, int e) {
        // Remove the initial check since we want to propagate
        // even zero values
        if(lazy[at].second == 1) { // If it's a set operation
            values[at] = lazy[at].first * (e-b+1); // **if
            // not sum then remove range
        }
        else { // If it's an update operation
            values[at] += lazy[at].first * (e-b+1); // if not
            // sum then remove range
        }
    }

    if(b != e) { // Not a leaf node
        if(lazy[at].second == 1) { // Set operation
            lazy[at*2] = lazy[at];
            lazy[at*2+1] = lazy[at];
        }
        else { // Update operation
            lazy[at*2].first += lazy[at].first;
            lazy[at*2+1].first += lazy[at].first;
        }
    }
    lazy[at] = {0,0}; // Always clear the lazy value
}

void update(int at,int b, int e, int l, int r,int v){
    push(at,b,e); // updating the sum at the node and
    // propagating pending update to the children

    if (r < b || e < l) return;

```

```
        if (l <= b && e <= r) {
            lazy[at].first += v;
            push(at,b,e);
            return;
        }
        int mid = (b+e)>>1;
        update(at*2,b,mid,l,r,v);
        update(at*2+1,mid+1,e,l,r,v);
        values[at] = merge(values[at*2],values[at*2+1]);
    }

    void set(int at,int b, int e, int l, int r,int v){
        push(at,b,e); // updating the sum at the node and
                        // propagating pending update to the children
        if (r < b || e < l) return;
        if (l <= b && e <= r) {
            lazy[at] = {v,1};
            push(at,b,e);
            return;
        }
        int mid = (b+e)>>1;
        set(at*2,b,mid,l,r,v);
        set(at*2+1,mid+1,e,l,r,v);
        values[at] = merge(values[at*2],values[at*2+1]);
    }

    //query(1,0,n-1,l,r) *** (l,r are 0 indedxed)
    int query(int at, int b, int e, int l, int r) {
        push(at,b,e); // updating the sum at the node
                        // and propagating pending update to the children
        if (b > r || e < l) {
            return 0;
        }
        if (l <= b && e <= r) {
            return values[at];
        }

        int mid = (b + e) >> 1;
        int left = query(at * 2, b, mid, l, r);
        int right = query(at * 2 + 1, mid + 1, e, l, r);
        return merge(left, right);
    }

    public:
    SegmentTree(vector<int>&a){
        n = a.size(); // **change
        values.resize(n*4);
        lazy.resize(n*4);
        build(a,1,0,n-1);
    }
    SegmentTree(int N){
        n = N;
        values.resize(n*4,0);
        lazy.resize(n*4);
    }
    int query(int l,int r){
        if(l>r)return 0; // ** change
        return query(1,0,n-1,l,r);
    }
    void update(int l,int r, int v){
        update(1,0,n-1,l,r,v);
    }
    void set(int l,int r,int v){
        set(1,0,n-1,l,r,v);
    }
};
```

```
PersistentSegmentTree.h
Description: Hash map with mostly the same API as unordered_map, but
~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if
provided).
b0b164, 76 lines

// modify as needed
struct item{
    int x;
};
item NUTRAL_ITEM = {0}; // merge(a , NUTRALITEM) = a
item merge(item a, item b){
    return {a.x+b.x};
}
item single(int v){
    return {v};
}
const int N = 2e5+5;
struct node{
    item v;
    int left;
    int right;
}t[N*22]; //Q log2(N)+N

class PersistentSegmentTree{
    vector<int>roots;
    int n;
    int id = 0;
    int newParent(int l, int r){
        int p = ++id;
        t[p].left = l;
        t[p].right = r;
        t[p].v = merge(t[l].v,t[r].v);
        return p;
    }
    int newNode(int v){
        int p = ++id;
        t[p].left = t[p].right = 0; //null
        t[p].v = single(v);
        return p;
    }
    int build(vector<int> &a,int b, int e){
        if(b == e) return newNode(a[b]);
        int mid = (b+e)>>1;
        return newParent( build(a,b,mid),build(a,mid+1,e) );
    }
    int update(int at,int b, int e, int i,int v){
        if (b == e && b == i) return newNode(v); //set value //
            // to increase value modify: newNode(v+t[at].v.x)
        int mid = (b+e)>>1;
        if(i<=mid)return newParent( update(t[at].left,b,mid,i,v)
            ,t[at].right);
        else return newParent( t[at].left,update(t[at].right,mid
            +1,e,i,v));
    }
    item query(int at, int b, int e, int l, int r) {
        if (at == 0 || b > r || e < l) return NUTRAL_ITEM;
        if (l <= b && e <= r) return t[at].v;
        int mid = (b + e) >> 1;
        item left = query(t[at].left, b, mid, l, r);
        item right = query(t[at].right, mid + 1, e, l, r);
        return merge(left, right);
    }

    public:
    PersistentSegmentTree(vector<int>&a){
        n = a.size();
        roots.push_back( build(a,0,n-1));
    }
    PersistentSegmentTree(int n){
```

```
        this->n = n;
        roots.push_back(newNode(0)); // blank node
        t[0].left = t[0].right = 0;
        t[0].v.x = 0;
    }
    item query(int version,int l,int r){
        return query(roots[version],0,n-1,l,r);
    }
    int update(int version,int i, int v){ // return
        // the version (index of the new root)
        int newRoot = update(roots[version],0,n-1,i,v);
        roots.push_back(newRoot);
        return roots.size()-1; // new version
    }
};
// initial verison = 0
```

UnionFind.h
Description: Disjoint-set data structure.
Time: O(α(N))
7aa27c, 14 lines

```
struct UF {
    vi e;
    UF(int n) : e(n, -1) {}
    bool sameSet(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        e[a] += e[b]; e[b] = a;
        return true;
    }
};
```

Matrix.h
Description: Basic operations on square matrices.
Usage: Matrix<int, 3> A;
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;
5ac17f, 45 lines

```
const int mod = 998244353;
struct Mat {
    int n, m;
    vector<vector<int>>> a;
    Mat() { }
    Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>>(
        m, 0)); }
    Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].
        size() : 0; a = v; }
    inline void make_unit() {
        assert(n == m);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) a[i][j] = i == j;
        }
    }
    inline Mat operator + (const Mat &b) {
        assert(n == b.n && m == b.m);
        Mat ans = Mat(n, m);
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
            }
        }
        return ans;
    }
    inline Mat operator * (const Mat &b) {
        assert(m == b.n);
        Mat ans = Mat(n, b.m);
```

```
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < b.m; j++) {
            for(int k = 0; k < m; k++) {
                ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j]
                    ) % mod) % mod;
            }}
        return ans;
    }
    inline Mat pow(long long k) {
        assert(n == m);
        Mat ans(n, n), t = a; ans.make_unit();
        while (k) {
            if (k & 1) ans = ans * t;
            t = t * t;
            k >>= 1;
        }
        return ans;}
    inline Mat& operator += (const Mat& b) { return *this = (*
        this) + b; }
    inline Mat& operator *= (const Mat& b) { return *this = (*
        this) * b; }
    inline bool operator == (const Mat& b) { return a == b.a; }
    inline bool operator != (const Mat& b) { return a != b.a; }
};
```

LineContainer.h
Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).
Time: $\mathcal{O}(\log N)$

Sec1c7, 30 lines

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

FenwickTree.h
Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

c980d1, 26 lines

```
template <class T>
struct BIT { //1-indexed
```

```
    int n; vector<T> t;
    BIT() {}
    BIT(int _n) {
        n = _n; t.assign(n + 1, 0);
    }
    T query(int i) {
        T ans = 0;
        for (; i >= 1; i -= (i & -i)) ans += t[i];
        return ans;
    }
    void upd(int i, T val) { //add T at i
        if (i <= 0) return;
        for (; i <= n; i += (i & -i)) t[i] += val;
    }
    void upd(int l, int r, T val) { // add T at all elements in [
        l,r]
        upd(l, val);
        upd(r + 1, -val);
    }
    T query(int l, int r) {
        return query(r) - query(l - 1);
    }
};
```

//initialize BIT<datatype>name(size);

RMQ.h
Description: Range Minimum Queries on an array. Returns $\min(V[a], V[a+1], \dots V[b-1])$ in constant time.
Usage: RMQ rmq(values);
rmq.query(inclusive, exclusive);
Time: $\mathcal{O}(|V| \log |V| + Q)$

510c32, 16 lines

```
template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j, 0, sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

MoQueries.h
Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).
Time: $\mathcal{O}(N\sqrt{Q})$

a12ef4, 49 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer
```

```
vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) {
        pii q = Q[qi];
```

```
        while (L > q.first) add(--L, 0);
        while (R < q.second) add(R++, 1);
        while (L < q.first) del(L++, 0);
        while (R > q.second) del(--R, 1);
        res[qi] = calc();
    }
    return res;
}
```

```
vi moTree(vector<array<int, 2>> Q, vector<vi& ed, int root=0){
    int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
    add(0, 0), in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto& f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep) I[x] = N++;
        for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
        if (!dep) I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);
    #define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) rep(end,0,2) {
        int &a = pos[end], b = Q[qi][end], i = 0;
    #define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
        else { add(c, end); in[c] = 1; } a = c; }
        while (!L[b] <= L[a] && R[a] <= R[b])
            I[i++] = b, b = par[b];
        while (a != b) step(par[a]);
        while (i--) step(I[i]);
        if (end) res[qi] = calc();
    }
    return res;
}
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

c9b7b0, 17 lines

```
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        rep(i,1,sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
```

PolyRoots.h
Description: Finds the real roots to a polynomial.
Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve $x^2-3x+2 = 0$
Time: $\mathcal{O}(n^2 \log(1/\epsilon))$
"Polynomial.h" b00bfe, 23 lines
vector<double> polyRoots(Poly p, double xmin, double xmax) {

```
    if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
    vector<double> ret;
    Poly der = p;
    der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.push_back(xmin-1);
    dr.push_back(xmax+1);
    sort(all(dr));
    rep(i,0,sz(dr)-1) {
        double l = dr[i], h = dr[i+1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            rep(it,0,60) { // while (h - l > 1e-8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign) l = m;
                else h = m;
            }
            ret.push_back((l + h) / 2);
        }
    }
    return ret;
}
```

PolyInterpolate.h

Description: Given n points $(x[i], y[i])$, computes an n -1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.
Time: $\mathcal{O}(n^2)$

08bf48, 13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    rep(k,0,n-1) rep(i,k+1,n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k,0,n) rep(i,0,n) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
Time: $\mathcal{O}(N^2)$

../number-theory/ModPow.h" 96548b, 20 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i,0,n) { ++m;
        ll d = s[i] % mod;
        rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * modpow(b, mod-2) % mod;
        rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
```

```
    for (ll& x : C) x = (mod - x) % mod;
    return C;
}
```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp-Massey.
Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number
Time: $\mathcal{O}(n^2 \log k)$

f4e444, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        rep(i,0,n+1) rep(j,0,n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep(j,0,n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is eps . Works equally well for maximization with a small change in the code. See Ternary-Search.h in the Various chapter for a discrete version.
Usage: double func(double x) { return 4+x+.3*x*x; }
double xmin = gss(-1000,1000,func);
Time: $\mathcal{O}(\log((b-a)/\epsilon))$

31d45b, 14 lines

```
double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}
```

8eeaf, 14 lines

HillClimbing.h

Description: Poor man's optimization for unimodal functions.

```
typedef array<double, 2> P;
```

```
template<class F> pair<double, P> hillClimb(P start, F f) {
    pair<double, P> cur(f(start), start);
    for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
        rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
            P p = cur.second;
            p[0] += dx*jmp;
            p[1] += dy*jmp;
            cur = min(cur, make_pair(f(p), p));
        }
    }
    return cur;
}
```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}(N^3)$

3313dc, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
    int n = sz(a); ll ans = 1;
    rep(i,0,n) {
        rep(j,i+1,n) {
            while (a[j][i] != 0) { // gcd step
                ll t = a[i][i] / a[j][i];
                if (t) rep(k,i,n)
                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2m)$

f94400, 35 lines

```
const int N = 3e5 + 9;

const double eps = 1e-9;
int Gauss(vector<vector<double>> a, vector<double> &ans) {
    int n = (int)a.size(), m = (int)a[0].size() - 1;
    vector<int> pos(m, -1);
```

```
double det = 1; int rank = 0;
for(int col = 0, row = 0; col < m && row < n; ++col) {
    int mx = row;
    for(int i = row; i < n; i++) if(fabs(a[i][col]) > fabs(a[mx][col])) mx = i;
    if(fabs(a[mx][col]) < eps) {det = 0; continue;}
    for(int i = col; i <= m; i++) swap(a[row][i], a[mx][i])
    ;
    if (row != mx) det = -det;
    det *= a[row][col];
    pos[col] = row;
    for(int i = 0; i < n; i++) {
        if(i != row && fabs(a[i][col]) > eps) {
            double c = a[i][col] / a[row][col];
            for(int j = col; j <= m; j++) a[i][j] -= a[row][j] * c;
        }
    }
    ++row; ++rank;
}
ans.assign(m, 0);
for(int i = 0; i < m; i++) {
    if(pos[i] != -1) ans[i] = a[pos[i]][m] / a[pos[i]][i];
}
for(int i = 0; i < n; i++) {
    double sum = 0;
    for(int j = 0; j < m; j++) sum += ans[j] * a[i][j];
    if(fabs(sum - a[i][m]) > eps) return -1; //no solution
}
for(int i = 0; i < m; i++) if(pos[i] == -1) return 2; //infinte solutions
return 1; //unique solution
}
```

SolveLinear2.h

Description: modular 585b19, 37 lines

```
int Gauss(vector<vector<int>>> a, vector<int> &ans){
    int n = a.size(), m = (int)a[0].size() - 1;
    vector<int> pos(m, -1);
    int free_var = 0;
    const long long MODSQ = (long long)mod * mod;
    int det = 1, rank = 0;
    for (int col = 0, row = 0; col < m && row < n; col++) {
        int mx = row;
        for (int k = row; k < n; k++) if (a[k][col] > a[mx][col]) mx = k;
        if (a[mx][col] == 0) {det = 0; continue;}
        for (int j = col; j <= m; j++) swap(a[mx][j], a[row][j]);
        if (row != mx) det = det == 0 ? 0 : mod - det;
        det = 1LL * det * a[row][col] % mod;
        pos[col] = row;
        int inv = power(a[row][col], mod - 2);
        for (int i = 0; i < n && inv; i++){
            if (i != row && a[i][col]) {
                int x = ((long long)a[i][col] * inv) % mod;
                for (int j = col; j <= m && x; j++){
                    if (a[row][j]) a[i][j] = (MODSQ + a[i][j] - ((long long)a[row][j] * x)) % mod;
                }
            }
        }
        row++; ++rank;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i++){
        if (pos[i] == -1) free_var++;
    }
}
```

SolveLinear2 SolveLinearBinary MatrixInverse

```
else ans[i] = ((long long)a[pos[i]][m] * power(a[pos[i]][i], mod - 2)) % mod;
}
for (int i = 0; i < n; i++) {
    long long val = 0;
    for (int j = 0; j < m; j++) val = (val + ((long long)ans[j] * a[i][j])) % mod;
    if (val != a[i][m]) return -1; //no solution
}
return free_var; //has solution
}
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b . Time: $\mathcal{O}(n^2m)$

78314c, 79 lines

```
const int N = 2010;

//n = number of equations, m = number of variables
int Gauss(int n, int m, vector<bitset<N>> a, bitset<N> &ans) {
    //reversing for lexicographically largest solution
    for (int i = 0; i < n; i++) {
        bitset<N> tmp;
        for (int j = 0; j < m; j++) tmp[j] = a[i][m - j - 1];
        tmp[m] = a[i][m];
        a[i] = tmp;
    }
    int rank = 0, det = 1;
    vector<int> pos(N, -1);
    for(int col = 0, row = 0; col < m && row < n; ++col) {
        int mx = row;
        for(int i = row; i < n; ++i) if(a[i][col]) { mx = i; break;
        }
        if(!a[mx][col]) { det = 0; continue; }
        swap(a[mx], a[row]);
        if(row != mx) det = det == 0 ? 0 : 1;
        det &= a[row][col];
        pos[col] = row;
        //forward elimination
        for(int i = row + 1; i < n; ++i) if (i != row && a[i][col])
            a[i] ^= a[row];
        ++row, ++rank;
    }
    ans.reset();
    //backward substitution
    for (int i = m - 1; i >= 0; i--) {
        if (pos[i] == -1) ans[i] = true;
        else {
            int k = pos[i];
            for (int j = i + 1; j < m; j++) if (a[k][j]) ans[i] = ans[i] ^ ans[j];
            ans[i] = ans[i] ^ a[k][m];
        }
    }
    for(int i = rank; i < n; ++i) if(a[i][m]) return -1; //no solution
    //reversing again beacuse we reversed earlier
    bitset<N> tmp;
    for (int j = 0; j < m; j++) tmp[j] = ans[m - j - 1];
    ans = tmp;
    int free_var = 0;
    for(int i = 0; i < m; ++i) if(pos[i] == -1) free_var++;
    return free_var; //has solution
}
string read() {
    string t;
    if(!(cin >> t)) return "";
    if(t.empty() || t == "and") return "";
}
```

```
while(t[0] == '(') t.erase(t.begin());
while(t.back() == ')') t.pop_back();
return t;
}
bool is_var(string t) { return t.size() > 0 && t[0] == 'x'; }
int get_var(string t) { return atoi(t.substr(1).c_str()) - 1; }
int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int n, m; cin >> n >> m;
    vector<bitset<N>> bs(n, bitset<N>(0));
    for(int i = 0; i < n; i++) {
        string s;
        bool eq = 1;
        while((s = read()).size() > 0) {
            if(is_var(s)) {
                int x = get_var(s);
                bs[i][x] = bs[i][x] ^ 1;
            }
            else if(s == "not") eq ^= 1;
        }
        bs[i][m] = eq;
    }
    bitset<N> ans;
    int ok = Gauss(n, m, bs, ans);
    if (ok == -1) cout << "impossible\n";
    else {
        for (int i = 0; i < m; i++) cout << "FT"[ans[i]]; cout << '\n';
    }
    return 0;
}
```

MatrixInverse.h

Description: Invert matrix A . Returns rank; result is stored in A unless singular ($\text{rank} < n$). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod{p}$, and k is doubled in each step. Time: $\mathcal{O}(n^3)$

ebfff6, 35 lines

```
int matInv(vector<vector<double>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;

    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n)
            if (fabs(A[j][k]) > fabs(A[r][c]))
                r = j, c = k;
        if (fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        rep(j,i+1,n) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] -= f*A[i][k];
            rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
        }
        rep(j,i+1,n) A[i][j] /= v;
        rep(j,0,n) tmp[i][j] /= v;
        A[i][i] = 1;
    }

    for (int i = n-1; i > 0; --i) rep(j,0,i) {
        double v = A[j][i];
    }
}
```

```
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
}

rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.

Time: $\mathcal{O}(N)$

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        } else {
            b[i] /= diag[i];
            if (i) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: `fft(a)` computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by `n`, `reverse(start+1, end)`, FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
```

```
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}

vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

NumberTheoreticTransform.h

Description: `ntt(a)` computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(\text{mod}-1)/N}$. N must be a power of 2. Useful for convolution modulo spefic nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by `n`, `reverse(start+1, end)`, NTT back. Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" ced03d, 35 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
}

vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
        n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i,0,n)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

464cf3, 16 lines

```
void FST(vi& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
                inv ? pii(v - u, u) : pii(v, u + v); // AND
                inv ? pii(v, u - v) : pii(u + v, u); // OR
                pii(u + v, u - v); // XOR
        }
    }
    if (inv) for (int& x : a) x /= sz(a); // XOR only
}

vi conv(vi a, vi b) {
    FST(a, 0); FST(b, 0);
    rep(i,0,sz(a)) a[i] *= b[i];
    FST(a, 1); return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

"euclid.h"	35bfea, 18 lines
<pre>const ll mod = 17; // change to something else struct Mod { ll x; Mod(ll xx) : x(xx) {} Mod operator+(Mod b) { return Mod((x + b.x) % mod); } Mod operator-(Mod b) { return Mod((x - b.x + mod) % mod); } Mod operator*(Mod b) { return Mod((x * b.x) % mod); } Mod operator/(Mod b) { return *this * invert(b); } Mod invert(Mod a) { ll x, y, g = euclid(a.x, mod, x, y); assert(g == 1); return Mod((x + mod) % mod); } Mod operator^(ll e) { if (!e) return Mod(1); Mod r = *this ^ (e / 2); r = r * r; return e&1 ? *this * r : r; } };</pre>	

ModInverse.h

Description: Pre-computation of modular inverses. Assumes LIM ≤ mod and that mod is a prime.

6f684f, 3 lines
<pre>const ll mod = 1000000007, LIM = 200000; ll* inv = new ll[LIM] - 1; inv[1] = 1; rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;</pre>

ModPow.h

b83e45, 8 lines
<pre>const ll mod = 1000000007; // faster if const</pre>

```
ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. $\text{modLog}(a,1,m)$ can be used to calculate the order of a .

c040b8, 11 lines
<pre>ll modLog(ll a, ll b, ll m) { ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1; unordered_map<ll, ll> A; while (j <= n && (e = f = e * a % m) != b % m) A[e * b % m] = j++; if (e == b % m) return j; if (__gcd(m, e) == __gcd(m, b)) rep(i,2,n+2) if (A.count(e = e * f % m)) return n * i - A[e]; return -1; }</pre>

ModSum.h

Description: Sums of mod'ed arithmetic progressions. $\text{modsum}(to, c, k, m) = \sum_{i=0}^{to-1} (ki+c)\%m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
```

```
ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$. **Time:** $\mathcal{O}(1)$ for modmul , $\mathcal{O}(\log b)$ for modpow

bbbd8f, 11 lines
<pre>typedef unsigned long long ull; ull modmul(ull a, ull b, ull M) { ll ret = a * b - M * ull(1.L / M * a * b); return ret + M * (ret < 0) - M * (ret >= (ll)M); } ull modpow(ull b, ull e, ull mod) { ull ans = 1; for (; e; b = modmul(b, b, mod), e /= 2) if (e & 1) ans = modmul(ans, b, mod); return ans; }</pre>

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution). **Time:** $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

19a793, 24 lines
"ModPow.h"
<pre>ll sqrt(ll a, ll p) { a %= p; if (a < 0) a += p; if (a == 0) return 0; assert(modpow(a, (p-1)/2, p) == 1); // else no solution if (p % 4 == 3) return modpow(a, (p+1)/4, p); // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5 ll s = p - 1, n = 2; int r = 0, m; while (s % 2 == 0) ++r, s /= 2; while (modpow(n, (p - 1) / 2, p) != p - 1) ++n; ll x = modpow(a, (s + 1) / 2, p); ll b = modpow(a, s, p), g = modpow(n, s, p); for (;;) r = m) { ll t = b; for (m = 0; m < r && t != 1; ++m) t = t * t % p; if (m == 0) return x; ll gs = modpow(g, 1LL << (r - m - 1), p); g = gs * gs % p; x = x * gs % p; b = b * g % p; } }</pre>

5.2 Primality

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \bmod c$.

60dcd1, 12 lines
"ModMulLL.h"
<pre>bool isPrime(ull n) { if (n < 2 n % 6 % 4 != 1) return (n 1) == 3; ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022}, s = __builtin_ctzll(n-1), d = n >> s; for (ull a : A) { // ^ count trailing zeroes ull p = modpow(a%n, d, n), i = s; while (p != 1 && p != n - 1 && a % n && i--) p = modmul(p, p, n); if (p != n-1 && i != s) return 0; } return 1; }</pre>

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}(n^{1/4})$, less for numbers with small factors.

d8d98d, 18 lines
"ModMulLL.h", "MillerRabin.h"
<pre>ull pollard(ull n) { ull x = 0, y = 0, t = 30, prd = 2, i = 1, q; auto f = [&](ull x) { return modmul(x, x, n) + i; }; while (t++ % 40 __gcd(prd, n) == 1) { if (x == y) x = ++i, y = f(x); if ((q = modmul(prd, max(x,y) - min(x,y), n)) prd = q; x = f(x), y = f(f(y))); } return __gcd(prd, n); } vector<ull> factor(ull n) { if (n == 1) return {}; if (isPrime(n)) return {n}; ull x = pollard(n); auto l = factor(x), r = factor(n / x); l.insert(l.end(), all(r)); return l; }</pre>

5.3 Divisibility

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need $\gcd$, use the built in $_gcd$ instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5 lines
<pre>ll euclid(ll a, ll b, ll &x, ll &y) { if (!b) return x = 1, y = 0, a; ll d = euclid(b, a % b, y, x); return y -= a/b * x, d; }</pre>

CRT.h

Description: Chinese Remainder Theorem.

$\text{crt}(a, m, b, n)$ computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

04d93a, 7 lines
"euclid.h"
<pre>ll crt(ll a, ll m, ll b, ll n) { if (n > m) swap(a, b), swap(m, n); ll x, y, g = euclid(m, n, x, y); assert((a - b) % g == 0); // else no solution x = (b - a) % n * x % n / g * m + a; return x < 0 ? x + m*n/g : x; }</pre>

5.3.1 Bézout’s identity

For $a \neq, b \neq 0$, then $d = gcd(a,b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x,y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{gcd(a,b)}, y - \frac{ka}{gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler’s ϕ function* is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2}...p_r^{k_r}$ then $\phi(n) = (p_1-1)p_1^{k_1-1}...(p_r-1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1-1/p)$. $\sum_{d|n} \phi(d) = n$, $\sum_{1 \leq k \leq n, gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler’s thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat’s little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \ \forall a$.

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

cf7d6d, 8 lines

5.4 Fractions

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p-1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$$\sum_{d|n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

phiFunction IntPerm multinomial

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

```
int permToInt(vi& v) {
    int use = 0, i = 0, r = 0;
    for (int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
        use |= 1 << x;
    return r;
}
```

044568, 6 lines

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k)\phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\sim 2e5$	$\sim 2e8$

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod p$.

6.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1!k_2!...k_n!}$.

```
ll multinomial(vi& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j+1);
    return c;
}
```

a0a312, 5 lines

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able). $B[0, \dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^\infty f(i) = \int_m^\infty f(x)dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m)$$

```
struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
};
```

```
int N;
vector<vector<edge>> ed;
vi seen;
vector<ll> dist, pi;
vector<edge*> par;

MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
    ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
}

void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({ 0, s });

    while (!q.empty()) {
        s = q.top().second; q.pop();
        seen[s] = 1; di = dist[s] + pi[s];
        for (edge& e : ed[s]) if (!seen[e.to]) {
            ll val = di - pi[e.to] + e.cost;
            if (e.cap - e.flow > 0 && val < dist[e.to]) {
                dist[e.to] = val;
                par[e.to] = &e;
                if (its[e.to] == q.end())
                    its[e.to] = q.push({ -dist[e.to], e.to });
                else
                    q.modify(its[e.to], { -dist[e.to], e.to });
            }
        }
    }
    rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        ll fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
    }
    rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
}

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i,0,N) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}
```

```
};

EdmondsKarp.h
Description: Flow algorithm with guaranteed complexity  $O(VE^2)$ . To get edge flow values, compare capacities before and after, and take the positive values only.
482fe0, 36 lines

template<class T> T edmondsKarp(vector<unordered_map<int, T>>&
    graph, int source, int sink) {
    assert(source != sink);
    T flow = 0;
    vi par(sz(graph)), q = par;

    for (;;) {
        fill(all(par), -1);
        par[source] = 0;
        int ptr = 1;
        q[0] = source;

        rep(i,0,ptr) {
            int x = q[i];
            for (auto e : graph[x]) {
                if (par[e.first] == -1 && e.second > 0) {
                    par[e.first] = x;
                    q[ptr++] = e.first;
                    if (e.first == sink) goto out;
                }
            }
        }
        return flow;
    out:
    T inc = numeric_limits<T>::max();
    for (int y = sink; y != source; y = par[y])
        inc = min(inc, graph[par[y]][y]);

    flow += inc;
    for (int y = sink; y != source; y = par[y]) {
        int p = par[y];
        if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
        graph[y][p] += inc;
    }
}

GlobalMinCut.h
Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time:  $O(V^3)$ 
8b0e19, 21 lines

pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    rep(i,0,n) co[i] = {i};
    rep(ph,1,n) {
        vi w = mat[0];
        size_t s = 0, t = 0;
        rep(it,0,n-ph) { //  $O(V^2) \rightarrow O(E \log V)$  with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            rep(i,0,n) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
    }
}
```

```
co[s].insert(co[s].end(), all(co[t]));
rep(i,0,n) mat[s][i] += mat[t][i];
rep(i,0,n) mat[i][s] = mat[s][i];
mat[0][t] = INT_MIN;
}
return best;
}
```

7.3 Matching

DFSMatching.h
Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: vi btoa(m, -1); dfsMatching(g, btoa);
Time: $O(VE)$
522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1; int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    return 0;
}

int dfsMatching(vector<vi>& g, vi& btoa) {
    vi vis;
    rep(i,0,sz(g)) {
        vis.assign(sz(btoa), 0);
        for (int j : g[i])
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
    return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

```
"DFSMatching.h"
da4196, 20 lines

vi cover(vector<vi>& g, int n, int m) {
    vi match(m, -1);
    int res = dfsMatching(g, match);
    vector<bool> lfound(n, true), seen(m);
    for (int it : match) if (it != -1) lfound[it] = false;
    vi q, cover;
    rep(i,0,n) if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back(); q.pop_back();
        lfound[i] = 1;
        for (int e : g[i]) if (!seen[e] && match[e] != -1) {
            seen[e] = true;
            q.push_back(match[e]);
        }
    }
    rep(i,0,n) if (!lfound[i]) cover.push_back(i);
    rep(i,0,m) if (seen[i]) cover.push_back(n+i);
    assert(sz(cover) == res);
    return cover;
}
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}(N^2M)$

1e0fe9, 31 lines

```
pair<int, vi> hungarian(const vector<vi> &a) {
    if (a.empty()) return {0, {}};
    int n = sz(a) + 1, m = sz(a[0]) + 1;
    vi u(n), v(m), p(m), ans(n - 1);
    rep(i,1,n) {
        p[0] = i;
        int j0 = 0; // add "dummy" worker 0
        vi dist(m, INT_MAX), pre(m, -1);
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1, delta = INT_MAX;
            rep(j,1,m) if (!done[j]) {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            rep(j,0,m) {
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
    }
    rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans}; // min cost
}
```

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod .
Time: $\mathcal{O}(N^3)$

cb1912, 40 lines

```
vector<pii> generalMatching(int N, vector<pii>& ed) {
    vector<vector<ll>> mat(N, vector<ll>(N)), A;
    for (pii pa : ed) {
        int a = pa.first, b = pa.second, r = rand() % mod;
        mat[a][b] = r, mat[b][a] = (mod - r) % mod;
    }

    int r = matInv(A = mat), M = 2*N - r, fi, fj;
    assert(r % 2 == 0);

    if (M != N) do {
        mat.resize(M, vector<ll>(M));
        rep(i,0,N) {
            mat[i].resize(M);
            rep(j,N,M) {
                int r = rand() % mod;
                mat[i][j] = r, mat[j][i] = (mod - r) % mod;
            }
        } while (matInv(A = mat) != M);

    vi has(M, 1); vector<pii> ret;
    rep(it,0,M/2) {
        rep(i,0,M) if (has[i])
            rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
```

```
        fi = i; fj = j; goto done;
    } assert(0); done:
    if (fj < N) ret.emplace_back(fi, fj);
    has[fi] = has[fj] = 0;
    rep(sw,0,2) {
        ll a = modpow(A[fi][fj], mod-2);
        rep(i,0,M) if (has[i] && A[i][fj]) {
            ll b = A[i][fj] * a % mod;
            rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod;
        }
        swap(fi,fj);
    }
    return ret;
}
```

7.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph.
Usage: N max number of nodes
G condensed SCC graph
vec nodes ordered by finish time
comp nodes in current SCC
dfs1 first DFS to compute order
dfs2 second DFS to collect SCC
idx component id of each node
scc total SCC count
needed.in SCCs with zero in degree
needed.out SCCs with zero out degree
ans edges needed to make graph one SCC
Time: $\mathcal{O}(E + V)$

5b5820, 62 lines

```
const int N = 3e5 + 9;

// given a directed graph return the minimum number of edges to
// be added so that the whole graph become an SCC
bool vis[N];
vector<int> g[N], r[N], G[N], vec; //G is the condensed graph
void dfs1(int u) {
    vis[u] = 1;
    for(auto v: g[u]) if(!vis[v]) dfs1(v);
    vec.push_back(u);
}

vector<int> comp;
void dfs2(int u) {
    comp.push_back(u);
    vis[u] = 1;
    for(auto v: r[u]) if(!vis[v]) dfs2(v);
}

int idx[N], in[N], out[N];
// idx: component id of each node.
// in/out: in/out degree of component
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int n, m;
    cin >> n >> m;
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        r[v].push_back(u); //reverse graph
    }
    for(int i = 1; i <= n; i++) if(!vis[i]) dfs1(i);
    reverse(vec.begin(), vec.end());
    memset(vis, 0, sizeof vis);
    int scc = 0; // # of connected components
```

```
for(auto u: vec) {
    if(!vis[u]) {
        comp.clear();
        dfs2(u);
        scc++;
        for(auto x: comp) idx[x]=scc;
    }
}
for(int u = 1; u <= n; u++) {
    for(auto v: g[u]) {
        if(idx[u] != idx[v]) {
            in[idx[v]]++, out[idx[u]]++;
            G[idx[u]].push_back(idx[v]);
        }
    }
}
int needed_in=0, needed_out=0;
for(int i = 1; i <= scc; i++) {
    if(!in[i]) needed_in++;
    if(!out[i]) needed_out++;
}
int ans = max(needed_in, needed_out);
if(scc == 1) ans = 0;
cout << ans << '\n';
return 0;
}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.
Usage: int eid = 0; ed.resize(N);
for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps[&] (const vi& edgelist) {...});
Time: $\mathcal{O}(E + V)$

c6b7c7, 32 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
    int me = num[at] = ++Time, top = me;
    for (auto [y, e] : ed[at]) if (e != par) {
        if (num[y]) {
            top = min(top, num[y]);
            if (num[y] < me)
                st.push_back(e);
        } else {
            int si = sz(st);
            int up = dfs(y, e, f);
            top = min(top, up);
            if (up == me) {
                st.push_back(e);
                f(vi(st.begin() + si, st.end()));
                st.resize(si);
            }
            else if (up < me) st.push_back(e);
            else { /* e is a bridge */ }
        }
    }
    return top;
}

template<class F>
void bicomps(F f) {
```

```
    num.assign(sz(ed), 0);
    rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V + E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>&& gr, int nedges, int src=0) {
    int n = sz(gr);
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        if (it == end){ ret.push_back(x); s.pop_back(); continue; }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1; s.push_back(y);
        }
    }
    for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
    return {ret.rbegin(), ret.rend()};
}
```

7.5 Coloring

7.6 Heuristics

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

bfce85, 25 lines

```
vector<vi> treeJump(vi& P){
    int on = 1, d = 1;
    while(on < sz(P)) on *= 2, d++;
    vector<vi> jmp(d, P);
    rep(i,1,d) rep(j,0,sz(P))
        jmp[i][j] = jmp[i-1][jmp[i-1][j]];
    return jmp;
}
```

```
int jmp(vector<vi>& tbl, int nod, int steps){
    rep(i,0,sz(tbl))
        if(steps&(1<<i)) nod = tbl[i][nod];
    return nod;
}
```

```
int lca(vector<vi>& tbl, vi& depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = sz(tbl); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d) a = c, b = d;
    }
    return tbl[0][a];
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time: $\mathcal{O}(N \log N + Q)$

14729d, 35 lines

```
const int N = 3e5 + 9, LG = 18;
vector<int> g[N];
int par[N][LG + 1], dep[N], sz[N];
void dfs(int u, int p = 0) {
    par[u][0] = p; dep[u] = dep[p] + 1; sz[u] = 1;
    for (int i = 1; i <= LG; i++) par[u][i] = par[par[u][i - 1]][i - 1];
    for (auto v : g[u]) if (v != p) {
        dfs(v, u); sz[u] += sz[v];
    }
}
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    for (int k = LG; k >= 0; k--) if (dep[par[u][k]] >= dep[v]) u = par[u][k];
    if (u == v) return u;
    for (int k = LG; k >= 0; k--) if (par[u][k] != par[v][k]) u = par[u][k], v = par[v][k];
    return par[u][0];
}
int kth(int u, int k) {
    assert(k >= 0);
    for (int i = 0; i <= LG; i++) if (k & (1 << i)) u = par[u][i];
    return u;
}
int dist(int u, int v) {
    int l = lca(u, v);
    return dep[u] + dep[v] - (dep[l] << 1);
}
//kth node from u to v, 0th node is u
int go(int u, int v, int k) {
    int l = lca(u, v);
    int d = dep[u] + dep[v] - (dep[l] << 1);
    assert(k <= d);
    if (dep[l] + k <= dep[u]) return kth(u, k);
    k -= dep[u] - dep[l];
    return kth(v, dep[v] - dep[l] - k);
}
```

1af152, 62 lines

VirtualTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(|S| \log |S|)$

```
const int N = 3e5 + 9;
vector<int> g[N];
int par[N][20], dep[N], sz[N], st[N], en[N], T;
void dfs(int u, int pre) {
    par[u][0] = pre;
    dep[u] = dep[pre] + 1;
    sz[u] = 1;
    st[u] = ++T;
    for (int i = 1; i <= 18; i++) par[u][i] = par[par[u][i - 1]][i - 1];
    for (auto v : g[u]) {
        if (v == pre) continue; dfs(v, u); sz[u] += sz[v];
    }
    en[u] = T;
}
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
```

```
    for (int k = 18; k >= 0; k--) if (dep[par[u][k]] >= dep[v]) u = par[u][k];
    if (u == v) return u;
    for (int k = 18; k >= 0; k--) if (par[u][k] != par[v][k]) u = par[u][k], v = par[v][k];
    return par[u][0];
}
int kth(int u, int k) {
    for (int i = 0; i <= 18; i++) if (k & (1 << i)) u = par[u][i];
    return u;
}
int dist(int u, int v) {
    int lc = lca(u, v);
    return dep[u] + dep[v] - 2 * dep[lc];
}
int isanc(int u, int v) {
    return (st[u] <= st[v]) && (en[v] <= en[u]);
}
vector<int> t[N];
// given specific nodes, construct a compressed directed tree
// with these vertices(if needed some other nodes included)
// returns the nodes of the tree
// nodes.front() is the root
// t[] is the specific tree
vector<int> buildtree(vector<int> v) {
    // sort by entry time
    sort(v.begin(), v.end(), [](int x, int y) {
        return st[x] < st[y];
    });
    // finding all the ancestors, there are few of them
    int s = v.size();
    for (int i = 0; i < s - 1; i++) {
        int lc = lca(v[i], v[i + 1]);
        v.push_back(lc);
    }
    // removing duplicated nodes
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    // again sort by entry time
    sort(v.begin(), v.end(), [](int x, int y) {
        return st[x] < st[y];
    });
    stack<int> st;
    st.push(v[0]);
    for (int i = 1; i < v.size(); i++) {
        while (!isanc(st.top(), v[i])) st.pop();
        t[st.top()].push_back(v[i]);
        st.push(v[i]);
    }
    return v;
}
int ans; int imp[N];
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
Time: $\mathcal{O}((\log N)^2)$

../data-structures/LazySegmentTree.h 9547af, 46 lines

```
template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    Node *tree;
    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
          rt(N), pos(N), tree(new Node(0, N)) { dfsSz(0); dfsHld(0); }
```

```
void dfsSz(int v) {
    for (int& u : adj[v]) {
        adj[u].erase(find(all(adj[u]), v));
        par[u] = v;
        dfsSz(u);
        siz[v] += siz[u];
        if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
}

void dfsHld(int v) {
    pos[v] = tim++;
    for (int u : adj[v]) {
        rt[u] = (u == adj[v][0] ? rt[v] : u);
        dfsHld(u);
    }
}

template <class B> void process(int u, int v, B op) {
    for (; v = par[rt[v]]) {
        if (pos[u] > pos[v]) swap(u, v);
        if (rt[u] == rt[v]) break;
        op(pos[rt[v]], pos[v] + 1);
    }
    op(pos[u] + VALS_EDGES, pos[v] + 1);
}

void modifyPath(int u, int v, int val) {
    process(u, v, [&](int l, int r) { tree->add(l, r, val); });
}

int queryPath(int u, int v) { // Modify depending on problem
    int res = -1e9;
    process(u, v, [&](int l, int r) {
        res = max(res, tree->query(l, r));
    });
    return res;
}

int querySubtree(int v) { // modifySubtree is similar
    return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
}
};
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $mat[a][b]--$, $mat[b][b]++$ (and $mat[b][a]--$, $mat[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Geo2D.h

```
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)
<bits/stdc++.h> d2c3b9, 937 lines

using namespace std;
// #define static_assert(...)

typedef double Tf;
typedef double Ti;
const Tf PI = acos(-1), EPS = 1e-9;
int dcmp(Tf x) { return abs(x) < EPS ? 0 : (x<0 ? -1 : 1); }

struct Point {
    Ti x, y;
    Point(Ti x = 0, Ti y = 0) : x(x), y(y) {}

    Point operator + (const Point& u) const { return Point(x + u.x, y + u.y); }
    Point operator - (const Point& u) const { return Point(x - u.x, y - u.y); }
    Point operator * (const long long u) const { return Point(x * u, y * u); }
    Point operator * (const Tf u) const { return Point(x * u, y * u); }
    Point operator / (const Tf u) const { return Point(x / u, y / u); }

    bool operator == (const Point& u) const { return dcmp(x - u.x) == 0 && dcmp(y - u.y) == 0; }
    bool operator != (const Point& u) const { return !(*this == u); }
    bool operator < (const Point& u) const { return dcmp(x - u.x) < 0 || (dcmp(x - u.x) == 0 && dcmp(y - u.y) < 0); }
    friend istream &operator >> (istream &is, Point &p) {
        return is >> p.x >> p.y;
    }
    friend ostream &operator << (ostream &os, const Point &p) {
        return os << p.x << " " << p.y;
    }
};

Ti dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
Ti cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
Tf length(Point a) { return sqrt(dot(a, a)); }
Ti sqLength(Point a) { return dot(a, a); }
Tf distance(Point a, Point b) { return length(a-b); }
Tf angle(Point u) { return atan2(u.y, u.x); }

// returns angle between oa, ob in (-PI, PI]
Tf angleBetween(Point a, Point b) {
    Tf ans = angle(b) - angle(a);
    return ans <= -PI ? ans + 2*PI : (ans > PI ? ans - 2*PI : ans);
}

// Rotate a ccw by rad radians
Point rotate(Point a, Tf rad) {
    static_assert(is_same<Tf, Ti>::value);
    return Point(a.x * cos(rad) - a.y * sin(rad), a.x * sin(rad) + a.y * cos(rad));
}

// rotate a ccw by angle th with cos(th) = co && sin(th) = si
Point rotatePrecise(Point a, Tf co, Tf si) {
    static_assert(is_same<Tf, Ti>::value);
    return Point(a.x * co - a.y * si, a.y * co + a.x * si);
}

Point rotate90(Point a) { return Point(-a.y, a.x); }

// scales vector a by s such that length of a becomes s
Point scale(Point a, Tf s) {
```

```
static_assert(is_same<Tf, Ti>::value);
return a / length(a) * s;
}

// returns an unit vector perpendicular to vector a
Point normal(Point a) {
    static_assert(is_same<Tf, Ti>::value);
    Tf l = length(a);
    return Point(-a.y / l, a.x / l);
}

// returns 1 if c is left of ab, 0 if on ab && -1 if right of ab
int orient(Point a, Point b, Point c) {
    return dcmp(cross(b - a, c - a));
}

struct polarComp {
    Point O, dir;
    polarComp(Point O = Point(0, 0), Point dir = Point(1, 0)) : O(O), dir(dir) {}
    bool half(Point p) {
        return dcmp(cross(dir, p)) < 0 || (dcmp(cross(dir, p)) == 0 && dcmp(dot(dir, p)) > 0);
    }
    bool operator() (Point p, Point q) {
        return make_tuple(half(p-O), 0) < make_tuple(half(q-O), cross(p-O, q-O));
    }
};

struct Segment {
    Point a, b;
    Segment() {}
    Segment(Point aa, Point bb) : a(aa), b(bb) {}
};
typedef Segment Line;

struct Circle {
    Point o;
    Tf r;
    Circle(Point o = Point(0, 0), Tf r = 0) : o(o), r(r) {}

    // returns true if point p is in || on the circle
    bool contains(Point p) {
        return dcmp(sqLength(p - o) - r * r) <= 0;
    }

    // returns a point on the circle rad radians away from +X CCW
    Point point(Tf rad) {
        static_assert(is_same<Tf, Ti>::value);
        return Point(o.x + cos(rad) * r, o.y + sin(rad) * r);
    }

    // area of a circular sector with central angle rad
    Tf area(Tf rad = PI + PI) { return rad * r * r / 2; }

    // area of the circular sector cut by a chord with central angle alpha
    Tf sector(Tf alpha) { return r * r * 0.5 * (alpha - sin(alpha)); }
};

namespace Linear {
    // returns true if point p is on segment s
    bool onSegment(Point p, Segment s) {
        return dcmp(cross(s.a - p, s.b - p)) == 0 && dcmp(dot(s.a - p, s.b - p)) <= 0;
    }
}
```

```

}

// returns true if segment p & q touch or intersect
bool segmentsIntersect(Segment p, Segment q) {
    if(onSegment(p.a, q) || onSegment(p.b, q)) return true;
    if(onSegment(q.a, p) || onSegment(q.b, p)) return true;

    Tf c1 = cross(p.b - p.a, q.a - p.a);
    Tf c2 = cross(p.b - p.a, q.b - p.a);
    Tf c3 = cross(q.b - q.a, p.a - q.a);
    Tf c4 = cross(q.b - q.a, p.b - q.a);
    return dcmp(c1) * dcmp(c2) < 0 && dcmp(c3) * dcmp(c4) < 0;
}

bool linesParallel(Line p, Line q) {
    return dcmp(cross(p.b - p.a, q.b - q.a)) == 0;
}

// lines are represented as a ray from a point: (point, vector)
// returns false if two lines (p, v) & (q, w) are parallel or collinear
// true otherwise, intersection point is stored at o via reference
bool lineLineIntersection(Point p, Point v, Point q, Point w, Point& o) {
    static_assert(is_same<Tf, Ti>::value);
    if(dcmp(cross(v, w)) == 0) return false;
    Point u = p - q;
    o = p + v * (cross(w, u) / cross(v, w));
    return true;
}

// returns false if two lines p & q are parallel or collinear
// true otherwise, intersection point is stored at o via reference
bool lineLineIntersection(Line p, Line q, Point& o) {
    return lineLineIntersection(p.a, p.b - p.a, q.a, q.b - q.a, o);
}

// returns the distance from point a to line l
Tf distancePointLine(Point p, Line l) {
    return abs(cross(l.b - l.a, p - l.a) / length(l.b - l.a));
}

// returns the shortest distance from point a to segment s
Tf distancePointSegment(Point p, Segment s) {
    if(s.a == s.b) return length(p - s.a);
    Point v1 = s.b - s.a, v2 = p - s.a, v3 = p - s.b;
    if(dcmp(dot(v1, v2)) < 0) return length(v2);
    else if(dcmp(dot(v1, v3)) > 0) return length(v3);
    else return abs(cross(v1, v2) / length(v1));
}

// returns the shortest distance from segment p to segment q
Tf distanceSegmentSegment(Segment p, Segment q) {
    if(segmentsIntersect(p, q)) return 0;
    Tf ans = distancePointSegment(p.a, q);
    ans = min(ans, distancePointSegment(p.b, q));
    ans = min(ans, distancePointSegment(q.a, p));
    ans = min(ans, distancePointSegment(q.b, p));
    return ans;
}

```

```

// returns the projection of point p on line l
Point projectPointLine(Point p, Line l) {
    static_assert(is_same<Tf, Ti>::value);
    Point v = l.b - l.a;
    return l.a + v * ((Tf) dot(v, p - l.a) / dot(v, v));
} // namespace Linear

typedef vector<Point> Polygon;
namespace Polygonal {
    // removes redundant colinear points
    // polygon can't be all colinear points
    Polygon RemoveCollinear(const Polygon& poly) {
        Polygon ret;
        int n = poly.size();
        for(int i = 0; i < n; i++) {
            Point a = poly[i];
            Point b = poly[(i + 1) % n];
            Point c = poly[(i + 2) % n];
            if(dcmp(cross(b-a, c-b)) != 0 && (ret.empty() || b != ret.back()))
                ret.push_back(b);
        }
        return ret;
    }

    // returns the signed area of polygon p of n vertices
    Tf signedPolygonArea(const Polygon& p) {
        Tf ret = 0;
        for(int i = 0; i < (int) p.size() - 1; i++)
            ret += cross(p[i]-p[0], p[i+1]-p[0]);
        return ret / 2;
    }

    // given a polygon p of n vertices, generates the convex hull in CCW
    // Tested on https://acm.timus.ru/problem.aspx?space=1&num=1185
    // Caution: when all points are colinear AND removeRedundant == false
    // output will be contain duplicate points (from upper hull) at back
    Polygon convexHull(Polygon p, bool removeRedundant) {
        int check = removeRedundant ? 0 : -1;
        sort(p.begin(), p.end());
        p.erase(unique(p.begin(), p.end()), p.end());

        int n = p.size();
        Polygon ch(n+n);
        int m = 0; // preparing lower hull
        for(int i = 0; i < n; i++) {
            while(m > 1 && dcmp(cross(ch[m-1] - ch[m-2], p[i] - ch[m-1])) <= check) m--;
            ch[m++] = p[i];
        }
        int k = m; // preparing upper hull
        for(int i = n-2; i >= 0; i--) {
            while(m > k && dcmp(cross(ch[m-1] - ch[m-2], p[i] - ch[m-2])) <= check) m--;
            ch[m++] = p[i];
        }
        if(n > 1) m--;
        ch.resize(m);
        return ch;
    }

    // Tested : https://www.spoj.com/problems/INOROUT
    // returns inside = -1, on = 0, outside = 1
    int pointInPolygon(const Polygon& p, Point o) {

```

```

using Linear::onSegment;
int wn = 0, n = p.size();
for(int i = 0; i < n; i++) {
    int j = (i + 1) % n;
    if(onSegment(o, Segment(p[i], p[j])) || o == p[i])
        return 0;
    int k = dcmp(cross(p[j] - p[i], o - p[i]));
    int d1 = dcmp(p[i].y - o.y);
    int d2 = dcmp(p[j].y - o.y);
    if(k > 0 && d1 <= 0 && d2 > 0) wn++;
    if(k < 0 && d2 <= 0 && d1 > 0) wn--;
}
return wn ? -1 : 1;
}

// Tested: Timus 1955, CF 598F
// Given a simple polygon p, and a line l, returns (x, y)
// x = total segment of l in p, y = total length of l in p.
pair<Tf, Tf> linePolygonIntersection(Line l, const Polygon& p) {
    using Linear::lineLineIntersection;
    int n = p.size();
    vector<pair<Tf, int>> ev;
    for(int i=0; i<n; ++i) {
        Point a = p[i], b = p[(i+1)%n], z = p[(i-1+n)%n];
        int ora = orient(l.a, l.b, a), orb = orient(l.a, l.b, b), orz = orient(l.a, l.b, z);
        if(!ora) {
            Tf d = dot(a - l.a, l.b - l.a);
            if(orz && orb) {
                if(orz != orb) ev.emplace_back(d, 0);
                //else // Point Touch
            }
            else if(orz) ev.emplace_back(d, orz);
            else if(orb) ev.emplace_back(d, orb);
        }
        else if(ora == -orb) {
            Point ins;
            lineLineIntersection(l, Line(a, b), ins);
            ev.emplace_back(dot(ins - l.a, l.b - l.a), 0);
        }
    }
    sort(ev.begin(), ev.end());

    Tf ans = 0, len = 0, last = 0, tot = 0;
    bool active = false;
    int sign = 0;
    for(auto &qq : ev) {
        int tp = qq.second;
        Tf d = qq.first;
        if(sign) {
            len += d-last; tot += d-last;
            ans = max(ans, len);
            if(tp != sign) active = !active;
            sign = 0;
        }
        else {
            if(active) {
                len += d-last; tot += d-last;
                ans = max(ans, len);
            }
            if(tp == 0) active = !active;
            else sign = tp;
        }
    }
    last = d;
    if(!active) len = 0;
}
ans /= length(l.b-l.a);

```

```

    tot /= length(l.b-l.a);
    return {ans, tot};
} // namespace Polygonal

namespace Convex {
    Polygon minkowskiSum(Polygon A, Polygon B){
        int n = A.size(), m = B.size();
        rotate(A.begin(), min_element(A.begin(), A.end()), A.end());
        rotate(B.begin(), min_element(B.begin(), B.end()), B.end());

        A.push_back(A[0]); B.push_back(B[0]);
        for(int i = 0; i < n; i++) A[i] = A[i+1] - A[i];
        for(int i = 0; i < m; i++) B[i] = B[i+1] - B[i];

        Polygon C(n+m+1);
        C[0] = A.back() + B.back();
        merge(A.begin(), A.end()-1, B.begin(), B.end()-1, C.begin()+1, polarComp(Point(0, 0), Point(0, -1)));
        for(int i = 1; i < C.size(); i++) C[i] = C[i] + C[i-1];
        C.pop_back();
        return C;
    }

    pair< Tf, Tf >rotatingCalipersBoundingBox(const Polygon &p)
    {
        using Linear::distancePointLine;
        static_assert(is_same<Tf, Ti>::value);
        int n = p.size();
        int l = 1, r = 1, j = 1;
        Tf area = 1e100;
        Tf perimeter = 1e100;
        for(int i = 0; i < n; i++) {
            Point v = (p[(i+1)%n] - p[i]) / length(p[(i+1)%n] - p[i]);
            while(dcmp(dot(v, p[r%n] - p[i]) - dot(v, p[(r+1)%n] - p[i])) < 0) r++;
            while(j < r || dcmp(cross(v, p[j%n] - p[i]) - cross(v, p[(j+1)%n] - p[i])) < 0) j++;
            while(l < j || dcmp(dot(v, p[l%n] - p[i]) - dot(v, p[(l+1)%n] - p[i])) > 0) l++;
            Tf w = dot(v, p[r%n] - p[i]) - dot(v, p[l%n] - p[i]);
            Tf h = distancePointLine(p[j%n], Line(p[i], p[(i+1)%n]));
            area = min(area, w * h);
            perimeter = min(perimeter, 2 * w + 2 * h);
        }
        return make_pair(area, perimeter);
    }

    // returns the left side of polygon u after cutting it by ray a->b
    Polygon cutPolygon(Polygon u, Point a, Point b) {
        using Linear::lineLineIntersection;
        using Linear::onSegment;

        Polygon ret;
        int n = u.size();
        for(int i = 0; i < n; i++) {
            Point c = u[i], d = u[(i + 1) % n];
            if(dcmp(cross(b-a, c-a)) >= 0) ret.push_back(c);
            if(dcmp(cross(b-a, d-c)) != 0) {
                Point t;
                lineLineIntersection(a, b - a, c, d - c, t);
                if(onSegment(t, Segment(c, d))) ret.push_back(t);
            }
        }
    }
}

```

```

    }
    return ret;
}

// returns true if point p is in or on triangle abc
bool pointInTriangle(Point a, Point b, Point c, Point p) {
    return dcmp(cross(b - a, p - a)) >= 0
        && dcmp(cross(c - b, p - b)) >= 0
        && dcmp(cross(a - c, p - c)) >= 0;
}

// Tested : https://www.spoj.com/problems/INOROUT
// pt must be in ccw order with no three collinear points
// returns inside = -1, on = 0, outside = 1
int pointInConvexPolygon(const Polygon &pt, Point p) {
    int n = pt.size();
    assert(n >= 3);

    int lo = 1, hi = n - 1;
    while(hi - lo > 1) {
        int mid = (lo + hi) / 2;
        if(dcmp(cross(pt[mid] - pt[0], p - pt[0])) > 0) lo = mid;
        else hi = mid;
    }

    bool in = pointInTriangle(pt[0], pt[lo], pt[hi], p);
    if(!in) return 1;

    if(dcmp(cross(pt[lo] - pt[lo - 1], p - pt[lo - 1])) == 0) return 0;
    if(dcmp(cross(pt[hi] - pt[lo], p - pt[lo])) == 0) return 0;
    if(dcmp(cross(pt[hi] - pt[(hi + 1) % n], p - pt[(hi + 1) % n])) == 0) return 0;
    return -1;
}

// Extreme Point for a direction is the farthest point in that direction
// also https://codeforces.com/blog/entry/48868
// u is the direction for extremeness
// weakly tested on https://open.kattis.com/problems/fenceortho
int extremePoint(const Polygon &poly, Point u) {
    int n = (int) poly.size();
    int a = 0, b = n;
    while(b - a > 1) {
        int c = (a + b) / 2;
        if(dcmp(dot(poly[c] - poly[(c + 1) % n], u)) >= 0
            && dcmp(dot(poly[c] - poly[(c - 1 + n) % n], u)) >= 0) {
            return c;
        }
    }

    bool a_up = dcmp(dot(poly[(a + 1) % n] - poly[a], u)) >= 0;
    bool c_up = dcmp(dot(poly[(c + 1) % n] - poly[c], u)) >= 0;
    bool a_above_c = dcmp(dot(poly[a] - poly[c], u)) > 0;

    if(a_up && !c_up) b = c;
    else if(!a_up && c_up) a = c;
    else if(a_up && c_up) {
        if(a_above_c) b = c;
        else a = c;
    }
}

```

```

    else {
        if(!a_above_c) b = c;
        else a = c;
    }
}

if(dcmp(dot(poly[a] - poly[(a + 1) % n], u)) > 0 && dcmp(dot(poly[a] - poly[(a - 1 + n) % n], u)) > 0)
    return a;
return b % n;
}

// For a convex polygon p and a line l, returns a list of segments
// of p that touch or intersect line l.
// the i'th segment is considered (p[i], p[(i + 1) modulo |p|])
// #1 If a segment is collinear with the line, only that is returned
// #2 Else if l goes through i'th point, the i'th segment is added
// Complexity: O(lg |p|)
vector<int> lineConvexPolyIntersection(const Polygon &p, Line l) {
    assert((int) p.size() >= 3);
    assert(l.a != l.b);

    int n = p.size();
    vector<int> ret;

    Point v = l.b - l.a;
    int lf = extremePoint(p, rotate90(v));
    int rt = extremePoint(p, rotate90(v) * Ti(-1));
    int olf = orient(l.a, l.b, p[lf]);
    int ort = orient(l.a, l.b, p[rt]);

    if(!olf || !ort) {
        int idx = (!olf ? lf : rt);
        if(orient(l.a, l.b, p[(idx - 1 + n) % n]) == 0)
            ret.push_back((idx - 1 + n) % n);
        else
            ret.push_back(idx);
        return ret;
    }
    if(olf == ort) return ret;

    for(int i=0; i<2; ++i) {
        int lo = i ? rt : lf;
        int hi = i ? lf : rt;
        int olo = i ? ort : olf;

        while(true) {
            int gap = (hi - lo + n) % n;
            if(gap < 2) break;

            int mid = (lo + gap / 2) % n;
            int omid = orient(l.a, l.b, p[mid]);
            if(!omid) {
                lo = mid;
                break;
            }
            if(omid == olo) lo = mid;
            else hi = mid;
        }
        ret.push_back(lo);
    }
    return ret;
}

// Tested : https://toph.co/p/cover-the-points

```

```

// Calculate [ACW, CW] tangent pair from an external point
constexpr int CW = -1, ACW = 1;
bool isGood(Point u, Point v, Point Q, int dir) { return
    orient(Q, u, v) != -dir; }
Point better(Point u, Point v, Point Q, int dir) { return
    orient(Q, u, v) == dir ? u : v; }
Point pointPolyTangent(const Polygon &pt, Point Q, int dir,
    int lo, int hi) {
    while(hi - lo > 1) {
        int mid = (lo + hi) / 2;
        bool pvs = isGood(pt[mid], pt[mid - 1], Q, dir);
        bool nxt = isGood(pt[mid], pt[mid + 1], Q, dir);

        if(pvs && nxt) return pt[mid];
        if(!pvs || !nxt) {
            Point p1 = pointPolyTangent(pt, Q, dir, mid +
                1, hi);
            Point p2 = pointPolyTangent(pt, Q, dir, lo, mid
                - 1);
            return better(p1, p2, Q, dir);
        }

        if(!pvs) {
            if(orient(Q, pt[mid], pt[lo]) == dir)
                hi = mid - 1;
            else if(better(pt[lo], pt[hi], Q, dir) == pt[lo])
                hi = mid - 1;
            else lo = mid + 1;
        }
        if(!nxt) {
            if(orient(Q, pt[mid], pt[lo]) == dir)
                lo = mid + 1;
            else if(better(pt[lo], pt[hi], Q, dir) == pt[lo])
                hi = mid - 1;
            else lo = mid + 1;
        }
    }

    Point ret = pt[lo];
    for(int i = lo + 1; i <= hi; i++) ret = better(ret, pt[i],
        Q, dir);
    return ret;
}

// [ACW, CW] Tangent
pair<Point, Point> pointPolyTangents(const Polygon &pt,
    Point Q) {
    int n = pt.size();
    Point acw_tan = pointPolyTangent(pt, Q, ACW, 0, n - 1);
    Point cw_tan = pointPolyTangent(pt, Q, CW, 0, n - 1);
    return make_pair(acw_tan, cw_tan);
}

namespace Circular {
    // Extremely inaccurate for finding near touches
    // compute intersection of line l with circle c
    // The intersections are given in order of the ray (l.a, l.
    // b)
    vector<Point> circleLineIntersection(Circle c, Line l) {
        static_assert(is_same<Tf, Ti>::value);
        vector<Point> ret;
        Point b = l.b - l.a, a = l.a - c.o;

        Tf A = dot(b, b), B = dot(a, b);
        Tf C = dot(a, a) - c.r * c.r, D = B*B - A*C;
        if (D < -EPS) return ret;

        ret.push_back(l.a + b * (-B - sqrt(D + EPS)) / A);
        if (D > EPS)

```

```

        ret.push_back(l.a + b * (-B + sqrt(D)) / A);
        return ret;
    }

    // signed area of intersection of circle(c.o, c.r) &&
    // triangle(c.o, s.a, s.b) [cross(a-o, b-o)/2]
    Tf circleTriangleIntersectionArea(Circle c, Segment s) {
        using Linear::distancePointSegment;
        Tf OA = length(c.o - s.a);
        Tf OB = length(c.o - s.b);

        // sector
        if(dcmp(distancePointSegment(c.o, s) - c.r) >= 0)
            return angleBetween(s.a-c.o, s.b-c.o) * (c.r * c.r)
                / 2.0;

        // triangle
        if(dcmp(OA - c.r) <= 0 && dcmp(OB - c.r) <= 0)
            return cross(c.o - s.b, s.a - s.b) / 2.0;

        // three part: (A, a) (a, b) (b, B)
        vector<Point> Sect = circleLineIntersection(c, s);
        return circleTriangleIntersectionArea(c, Segment(s.a,
            Sect[0]))
            + circleTriangleIntersectionArea(c, Segment(Sect
                [0], Sect[1]))
            + circleTriangleIntersectionArea(c, Segment(Sect
                [1], s.b));
    }

    // area of intersecion of circle(c.o, c.r) && simple
    // polyson(p[])
    // Tested : https://codeforces.com/gym/100204/problem/F -
    // Little Mammoth
    Tf circlePolyIntersectionArea(Circle c, Polygon p) {
        Tf res = 0;
        int n = p.size();
        for(int i = 0; i < n; ++i)
            res += circleTriangleIntersectionArea(c, Segment(p[i],
                p[(i + 1) % n]));
        return abs(res);
    }

    // locates circle c2 relative to c1
    // interior (d < R - r) ———> -2
    // interior tangents (d = R - r) ———> -1
    // concentric (d = 0)
    // secants (R - r < d < R + r) ———> 0
    // exterior tangents (d = R + r) ———> 1
    // exterior (d > R + r) ———> 2
    int circleCirclePosition(Circle c1, Circle c2) {
        Tf d = length(c1.o - c2.o);
        int in = dcmp(d - abs(c1.r - c2.r)), ex = dcmp(d - (c1.
            r + c2.r));
        return in < 0 ? -2 : in == 0 ? -1 : ex == 0 ? 1 : ex >
            0 ? 2 : 0;
    }

    // compute the intersection points between two circles c1
    // && c2
    vector<Point> circleCircleIntersection(Circle c1, Circle c2
        ) {
        static_assert(is_same<Tf, Ti>::value);

        vector<Point> ret;
        Tf d = length(c1.o - c2.o);
        if(dcmp(d) == 0) return ret;
        if(dcmp(c1.r + c2.r - d) < 0) return ret;
        if(dcmp(abs(c1.r - c2.r) - d) > 0) return ret;

```

```

        Point v = c2.o - c1.o;
        Tf co = (c1.r * c1.r + sqLength(v) - c2.r * c2.r) / (2
            * c1.r * length(v));
        Tf si = sqrt(abs(1.0 - co * co));
        Point p1 = scale(rotatePrecise(v, co, -si), c1.r) + c1.
            o;
        Point p2 = scale(rotatePrecise(v, co, si), c1.r) + c1.o
            ;

        ret.push_back(p1);
        if(p1 != p2) ret.push_back(p2);
        return ret;
    }

    // intersection area between two circles c1, c2
    Tf circleCircleIntersectionArea(Circle c1, Circle c2) {
        Point AB = c2.o - c1.o;
        Tf d = length(AB);
        if(d >= c1.r + c2.r) return 0;
        if(d + c1.r <= c2.r) return PI * c1.r * c1.r;
        if(d + c2.r <= c1.r) return PI * c2.r * c2.r;

        Tf alpha1 = acos((c1.r * c1.r + d * d - c2.r * c2.r) /
            (2.0 * c1.r * d));
        Tf alpha2 = acos((c2.r * c2.r + d * d - c1.r * c1.r) /
            (2.0 * c2.r * d));
        return c1.sector(2 * alpha1) + c2.sector(2 * alpha2);
    }

    // returns tangents from a point p to circle c
    vector<Point> pointCircleTangents(Point p, Circle c) {
        static_assert(is_same<Tf, Ti>::value);

        vector<Point> ret;
        Point u = c.o - p;
        Tf d = length(u);
        if(d < c.r) ;
        else if(dcmp(d - c.r) == 0) {
            ret = { rotate(u, PI / 2) };
        }
        else {
            Tf ang = asin(c.r / d);
            ret = { rotate(u, -ang), rotate(u, ang) };
        }
        return ret;
    }

    // returns the points on tangents that touches the circle
    vector<Point> pointCircleTangencyPoints(Point p, Circle c)
    {
        static_assert(is_same<Tf, Ti>::value);

        Point u = p - c.o;
        Tf d = length(u);
        if(d < c.r) return {};
        else if(dcmp(d - c.r) == 0) return {c.o + u};
        else {
            Tf ang = acos(c.r / d);
            u = u / length(u) * c.r;
            return { c.o + rotate(u, -ang), c.o + rotate(u, ang)
                };
        }
    }

    // for two circles c1 && c2, returns two list of points a
    // && b
    // such that a[i] is on c1 && b[i] is c2 && for every i
    // Line(a[i], b[i]) is a tangent to both circles

```

```
// CAUTION: a[i] = b[i] in case they touch | -1 for c1 = c2
int circleCircleTangencyPoints(Circle c1, Circle c2, vector
<Point> &a, vector<Point> &b) {
    a.clear(), b.clear();
    int cnt = 0;
    if(dcmp(c1.r - c2.r) < 0) {
        swap(c1, c2); swap(a, b);
    }
    Tf d2 = sqLength(c1.o - c2.o);
    Tf rdif = c1.r - c2.r, rsum = c1.r + c2.r;
    if(dcmp(d2 - rdif * rdif) < 0) return 0;
    if(dcmp(d2) == 0 && dcmp(c1.r - c2.r) == 0) return -1;

    Tf base = angle(c2.o - c1.o);
    if(dcmp(d2 - rdif * rdif) == 0) {
        a.push_back(c1.point(base));
        b.push_back(c2.point(base));
        cnt++;
        return cnt;
    }

    Tf ang = acos((c1.r - c2.r) / sqrt(d2));
    a.push_back(c1.point(base + ang));
    b.push_back(c2.point(base + ang));
    cnt++;
    a.push_back(c1.point(base - ang));
    b.push_back(c2.point(base - ang));
    cnt++;

    if(dcmp(d2 - rsum * rsum) == 0) {
        a.push_back(c1.point(base));
        b.push_back(c2.point(PI + base));
        cnt++;
    }
    else if(dcmp(d2 - rsum * rsum) > 0) {
        Tf ang = acos((c1.r + c2.r) / sqrt(d2));
        a.push_back(c1.point(base + ang));
        b.push_back(c2.point(PI + base + ang));
        cnt++;
        a.push_back(c1.point(base - ang));
        b.push_back(c2.point(PI + base - ang));
        cnt++;
    }
    return cnt;
}

// namespace Circular
namespace EnclosingCircle{
    // returns false if points are collinear, true otherwise
    // circle p touch each arm of triangle abc
    // circle p touch each arm of triangle abc
    bool inCircle(Point a, Point b, Point c, Circle &p) {
        using Linear::distancePointLine;
        static_assert(is_same<Tf, Ti>::value);
        if(orient(a, b, c) == 0) return false;
        Tf u = length(b - c);
        Tf v = length(c - a);
        Tf w = length(a - b);
        p.o = (a * u + b * v + c * w) / (u + v + w);
        p.r = distancePointLine(p.o, Line(a, b));
        return true;
    }

    // set of points A(x, y) such that PA : QA = rp : rq
    Circle apolloniusCircle(Point P, Point Q, Tf rp, Tf rq) {
        static_assert(is_same<Tf, Ti>::value);
        rq *= rq; rp *= rp;
        Tf a = rq - rp;
        assert(dcmp(a));

```

```
Tf g = (rq*P.x - rp*Q.x)/a;
Tf h = (rq*P.y - rp*Q.y)/a;
Tf c = (rq*P.x*P.x - rp*Q.x*Q.x + rq*P.y*P.y - rp*Q.y*Q.y)/a;
Point o(g, h);
Tf R = sqrt(g * g + h * h - c);
return Circle(o, R);
}

// returns false if points are collinear, true otherwise
// circle p goes through points a, b && c
bool circumCircle(Point a, Point b, Point c, Circle &p) {
    using Linear::lineLineIntersection;
    if(orient(a, b, c) == 0) return false;
    Point d = (a + b) / 2, e = (a + c) / 2;
    Point vd = rotate90(b - a), ve = rotate90(a - c);
    bool f = lineLineIntersection(d, vd, e, ve, p.o);
    if(f) p.r = length(a - p.o);
    return f;
}

Circle boundary(const vector<Point> &p) {
    Circle ret;
    int sz = p.size();
    if(sz == 0) ret.r = 0;
    else if(sz == 1) ret.o = p[0], ret.r = 0;
    else if(sz == 2) ret.o = (p[0] + p[1]) / 2, ret.r =
        length(p[0] - p[1]) / 2;
    else if(!circumCircle(p[0], p[1], p[2], ret)) ret.r
        = 0;
    return ret;
}

Circle welzl(const vector<Point> &p, int fr, vector<Point>
&b) {
    if(fr >= (int) p.size() || b.size() == 3) return
        boundary(b);

    Circle c = welzl(p, fr + 1, b);
    if(!c.contains(p[fr])) {
        b.push_back(p[fr]);
        c = welzl(p, fr + 1, b);
        b.pop_back();
    }
    return c;
}

Circle MEC(vector<Point> p) {
    random_shuffle(p.begin(), p.end());
    vector<Point> q;
    return welzl(p, 0, q);
}

// Given a list of segments v, finds a pair (i, j)
// st v[i], v[j] intersects. If none, returns {-1, -1}
// Tested Timus 1469, CF 1359F
namespace IntersectingSegments {
    struct Event {
        Tf x;
        int tp, id;
        bool operator < (const Event &p) const {
            if(dcmp(x - p.x)) return x < p.x;
            return tp > p.tp;
        }
    };

    pair<int, int> anyIntersection(const vector<Segment> &v) {
        using Linear::segmentsIntersect;
        static_assert(is_same<Tf, Ti>::value);

```

```
vector<Event> ev;
for(int i=0; i<v.size(); i++) {
    ev.push_back({min(v[i].a.x, v[i].b.x), +1, i});
    ev.push_back({max(v[i].a.x, v[i].b.x), -1, i});
}
sort(ev.begin(), ev.end());

auto comp = [&v] (int i, int j) {
    Segment p = v[i], q = v[j];
    Tf x = max(min(p.a.x, p.b.x), min(q.a.x, q.b.x));
    auto yvalSegment = [&x] (const Line &s) {
        if(dcmp(s.a.x - s.b.x) == 0) return s.a.y;
        return s.a.y + (s.b.y - s.a.y) * (x - s.a.x) /
            (s.b.x - s.a.x);
    };
    return dcmp(yvalSegment(p) - yvalSegment(q)) < 0;
};

multiset<int, decltype(comp)> st(comp);
typedef decltype(st)::iterator iter;
auto prev = [&st](iter it) {
    return it == st.begin() ? st.end() : --it;
};
auto next = [&st](iter it) {
    return it == st.end() ? st.end() : ++it;
};

vector<iter> pos(v.size());
for(auto &cur : ev) {
    int id = cur.id;
    if(cur.tp == 1) {
        iter nxt = st.lower_bound(id);
        iter pre = prev(nxt);
        if(pre != st.end() && segmentsIntersect(v[*pre], v[id])) return {*pre, id};
        if(nxt != st.end() && segmentsIntersect(v[*nxt], v[id])) return {*nxt, id};
        pos[id] = st.insert(nxt, id);
    }
    else {
        iter nxt = next(pos[id]);
        iter pre = prev(pos[id]);
        if(pre != st.end() && nxt != st.end() &&
            segmentsIntersect(v[*pre], v[*nxt]))
            return {*pre, *nxt};
        st.erase(pos[id]);
    }
}
return {-1, -1};
}

namespace HalfPlanar {
    using Linear::lineLineIntersection;
    struct DirLine {
        Point p, v;
        Tf ang;
        DirLine() {}
        DirLine(Point p, Point v) : p(p), v(v) { ang = atan2(v.y, v.x); }
        DirLine(Tf a, Tf b, Tf c) {
            assert(dcmp(a) || dcmp(b));
            p = dcmp(a) ? Point(-c/a, 0) : Point(0, -c/b);
            v = Point(b, -a);
            ang = atan2(v.y, v.x);
        }
        bool operator<(const DirLine& u) const { return ang < u.ang; }
    };

```

```

    bool onLeft(Point x) const { return dcmp(cross(v, x-p))
        >= 0; }

};

// Returns the region bounded by the left side of some
// directed lines
// MAY CONTAIN DUPLICATE POINTS
// OUTPUT IS UNDEFINED if intersection is unbounded
// Complexity: O(n log n) for sorting, O(n) afterwards
Polygon halfPlaneIntersection(vector<DirLine> li) {
    int n = li.size(), first = 0, last = 0;
    sort(li.begin(), li.end());
    vector<Point> p(n);
    vector<DirLine> q(n);
    q[0] = li[0];

    for(int i = 1; i < n; i++) {
        while(first < last && !li[i].onLeft(p[last - 1]))
            last--;
        while(first < last && !li[i].onLeft(p[first]))
            first++;
        q[++last] = li[i];
        if(dcmp(cross(q[last].v, q[last-1].v)) == 0) {
            last--;
            if(q[last].onLeft(li[i].p)) q[last] = li[i];
        }
        if(first < last)
            lineLineIntersection(q[last - 1].p, q[last -
                1].v, q[last].p, q[last].v, p[last - 1]);
    }

    while(first < last && !q[first].onLeft(p[last - 1]))
        last--;
    if(last - first <= 1) return {};
    lineLineIntersection(q[last].p, q[last].v, q[first].p,
        q[first].v, p[last]);
    return Polygon(p.begin()+first, p.begin()+last+1);
}

// O(n^2 lg n) implementation of Voronoi Diagram bounded by
// INF square
// returns region, where regions[i] = set of points for
// which closest
// point is site[i]. This region is a polygon.
const Tf INF = 1e10;
vector<Polygon> voronoi(const vector<Point> &site, Tf bsq)
{
    int n = site.size();
    vector<Polygon> region(n);
    Point A(-bsq, -bsq), B(bsq, -bsq), C(bsq, bsq), D(-bsq,
        bsq);

    for(int i = 0; i < n; ++i) {
        vector<DirLine> li(n - 1);
        for(int j = 0, k = 0; j < n; ++j) {
            if(i == j) continue;
            li[k++] = DirLine((site[i] + site[j]) / 2,
                rotate90(site[j] - site[i]));
        }
        li.emplace_back(A, B - A);
        li.emplace_back(B, C - B);
        li.emplace_back(C, D - C);
        li.emplace_back(D, A - D);
        region[i] = halfPlaneIntersection(li);
    }
    return region;
}
}

```

```

namespace PointRotationTrick {
    typedef pair< int , int >PII;
    void performTrick(vector< Point >pts, const function<void(
        const vector< Point >&, int)> &processor) {
        int n = pts.size();
        sort(pts.begin(), pts.end());
        vector< int >position(n);
        vector< PII >segments;
        segments.reserve((n*(n-1))/2);
        for (int i = 0; i < n; i++) {
            position[i] = i;
            for (int j = i+1; j < n; j++) {
                segments.emplace_back(i, j);
            }
        }
        assert(segments.capacity() == segments.size());
        sort(segments.begin(), segments.end(), [&](PII p, PII q
            ) {
                Ti prod = cross(pts[p.second]-pts[p.first], pts[q.
                    second]-pts[q.first]);
                if (prod != 0) return prod > 0;
                return p < q;
            });
        for (PII seg : segments) {
            int i = position[seg.first];
            assert(position[seg.second] == i+1);
            processor(pts, i);
            swap(pts[i], pts[i+1]);
            swap(position[seg.first], position[seg.second]);
        }
    }
}

```

Geo3D.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

<bits/stdc++.h> b46b38, 831 lines

using namespace std;

// https://victorlecomte.com/cp-geo.pdf

```

const double inf = 1e100;
const double eps = 1e-9;
const double PI = acos((double)-1.0);
int sign(double x) { return (x > eps) - (x < -eps); }

```

```

struct PT {
    double x, y;
    PT() { x = 0, y = 0; }
    PT(double x, double y) : x(x), y(y) {}
    PT(const PT &p) : x(p.x), y(p.y) {}
    void scan() { cin >> x >> y; }
    PT operator + (const PT &a) const { return PT(x + a.x, y +
        a.y); }
    PT operator - (const PT &a) const { return PT(x - a.x, y -
        a.y); }
    PT operator * (const double a) const { return PT(x * a, y *
        a); }
    friend PT operator * (const double &a, const PT &b) {
        return PT(a * b.x, a * b.y); }
    PT operator / (const double a) const { return PT(x / a, y /
        a); }
    bool operator == (PT a) const { return sign(a.x - x) == 0
        && sign(a.y - y) == 0; }
    bool operator != (PT a) const { return !(*this == a); }
    bool operator < (PT a) const { return sign(a.x - x) == 0 ?
        y < a.y : x < a.x; }
    bool operator > (PT a) const { return sign(a.x - x) == 0 ?
        y > a.y : x > a.x; }
    double norm() { return sqrt(x * x + y * y); }
}

```

```

double norm2() { return x * x + y * y; }
PT perp() { return PT(-y, x); }
double arg() { return atan2(y, x); }
PT truncate(double r) { // returns a vector with norm r and
    having same direction
    double k = norm();
    if (!sign(k)) return *this;
    r /= k;
    return PT(x * r, y * r);
}

};
inline double dot(PT a, PT b) { return a.x * b.x + a.y * b.y; }
inline double dist2(PT a, PT b) { return dot(a - b, a - b); }
inline double dist(PT a, PT b) { return sqrt(dot(a - b, a - b))
    ; }
inline double cross(PT a, PT b) { return a.x * b.y - a.y * b.x;
    }
inline int orientation(PT a, PT b, PT c) { return sign(cross(b
    - a, c - a)); }
PT perp(PT a) { return PT(-a.y, a.x); }
PT rotateccw90(PT a) { return PT(-a.y, a.x); }
PT rotatecw90(PT a) { return PT(a.y, -a.x); }
PT rotateccw(PT a, double t) { return PT(a.x * cos(t) - a.y *
    sin(t), a.x * sin(t) + a.y * cos(t)); }
PT rotatecw(PT a, double t) { return PT(a.x * cos(t) + a.y *
    sin(t), -a.x * sin(t) + a.y * cos(t)); }
double SQ(double x) { return x * x; }
double rad_to_deg(double r) { return (r * 180.0 / PI); }
double deg_to_rad(double d) { return (d * PI / 180.0); }
double get_angle(PT a, PT b) {
    double costheta = dot(a, b) / a.norm() / b.norm();
    return acos(max((double)-1.0, min((double)1.0, costheta)));
}
struct p3 {
    double x, y, z;
    p3() { x = 0, y = 0; z = 0; }
    p3(double x, double y, double z) : x(x), y(y), z(z) {}
    p3(const p3 &p) : x(p.x), y(p.y), z(p.z) {}
    void scan() { cin >> x >> y >> z; }
    p3 operator + (const p3 &a) const { return p3(x + a.x, y +
        a.y, z + a.z); }
    p3 operator - (const p3 &a) const { return p3(x - a.x, y -
        a.y, z - a.z); }
    p3 operator * (const double a) const { return p3(x * a, y *
        a, z * a); }
    friend p3 operator * (const double &a, const p3 &b) {
        return p3(a * b.x, a * b.y, a * b.z); }
    p3 operator / (const double a) const { return p3(x / a, y /
        a, z / a); }
    bool operator == (p3 a) const { return sign(a.x - x) == 0
        && sign(a.y - y) == 0 && sign(a.z - z) == 0; }
    bool operator != (p3 a) const { return !(*this == a); }
    double abs() { return sqrt(x * x + y * y + z * z); }
    double sq() { return x * x + y * y + z * z; }
    p3 unit() { return *this / abs(); }
}zero(0, 0, 0);
double operator | (p3 v, p3 w) { //dot product
    return v.x * w.x + v.y * w.y + v.z * w.z;
}
p3 operator * (p3 v, p3 w) { //cross product
    return {v.y * w.z - v.z * w.y, v.z * w.x - v.x * w.z, v.x *
        w.y - v.y * w.x};
}
double sq(p3 v) { return v | v; }
double abs(p3 v) { return sqrt(sq(v)); }
p3 unit(p3 v) { return v / abs(v); }
inline double dot(p3 a, p3 b) { return a.x * b.x + a.y * b.y +
    a.z * b.z; }
inline double dist2(p3 a, p3 b) { return dot(a - b, a - b); }

```

```

inline double dist(p3 a, p3 b) { return sqrt(dot(a - b, a - b))
; }
inline p3 cross(p3 a, p3 b) { return p3(a.y * b.z - a.z * b.y,
    a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x); }
// if s is on the same side of the plane pqr as the vector pq *
// pr then it will be positive
// otherwise negative or 0 if on the plane
double orient(p3 p, p3 q, p3 r, p3 s) { return (q - p) * (r - p)
    | (s - p); }
// returns orientation of p to q to r on the plane
// perpendicular to n
// assuming p, q, r are on the plane
double orient_by_normal(p3 p, p3 q, p3 r, p3 n) { return (q - p)
    * (r - p) | n; }
double get_angle(p3 a, p3 b) {
    double costheta = dot(a, b) / a.abs() / b.abs();
    return acos(max((double)-1.0, min((double)1.0, costheta)));
}
double small_angle(p3 v, p3 w) {
    return acos(min(fabs(v | w) / abs(v) / abs(w), (double)1.0)
    );
}

struct plane {
    // n is the perpendicular normal vector to the plane
    p3 n; double d; // (n | p) = d
    // From normal n and offset d
    plane(p3 n, double d) : n(n), d(d) {}
    // From normal n and point P
    plane(p3 n, p3 p) : n(n), d(n | p) {}
    // From three non-collinear points P,Q,R
    plane(p3 p, p3 q, p3 r) : plane((q - p) * (r - p), p) {}
    // positive if on the same side as the normal, negative if
    // on the opposite side, 0 if on the plane
    double side(p3 p) { return (n | p) - d; }
    // distance from point p to plane
    double dist(p3 p) { return fabs(side(p)) / abs(n); }
    // translate the plane by vector t
    plane translate(p3 t) { return {n, d + (n | t)}; }
    // shift the plane perpendicular to n by distance dist
    plane shiftUp(double dist) { return {n, d + dist * abs(n)}; }
}
// orthogonal projection of point p onto plane
p3 proj(p3 p) { return p - n * side(p) / sq(n); }
// orthogonal reflection of point p onto plane
p3 refl(p3 p) { return p - n * 2 * side(p) / sq(n); }
pair<p3, p3> get_two_points_on_plane() {
    assert(sign(n.x) != 0 || sign(n.y) != 0 || sign(n.z) != 0);
    if (sign(n.x) == 0 && sign(n.y) == 0) return {p3(1, 0,
        d/n.z), p3(0, 1, d/n.z)};
    if (sign(n.y) == 0 && sign(n.z) == 0) return {p3(d/n.x,
        1, 0), p3(d/n.x, 0, 1)};
    if (sign(n.z) == 0 && sign(n.x) == 0) return {p3(1, d/n
        .y, 0), p3(0, d/n.y, 1)};
    if (sign(n.x) == 0) return {p3(1, d/n.y, 0), p3(0, 0,
        d/n.z)};
    if (sign(n.y) == 0) return {p3(0, 1, d/n.z), p3(d/n.x,
        0, 0)};
    if (sign(n.z) == 0) return {p3(d/n.x, 0, 1), p3(0, d/n
        .y, 0)};
    if (sign(d)!=0) return {p3(d/n.x, 0, 0), p3(0, d/n.y,
        0)};
    return {p3(n.y, -n.x, 0), p3(-n.y, n.x, 0)};
}

struct coords {
    // coordinate system for coplanar points

```

```

// o is the origin, dx, dy, dz are unit vectors similar to
// normal 3D system
// but dx and dy are on the plane
p3 o, dx, dy, dz;
// From three points P, Q, R on the plane
coords(p3 p, p3 q, p3 r) : o(p) {
    dx = unit(q - p);
    dz = unit(dx * (r - p));
    dy = dz * dx;
}
// From four points P,Q,R,S: take directions PQ, PR, PS as
// is
// it allows us to keep using integer coordinates but has
// some pitfalls
// e.g. distances and angles are not preserved but relative
// positions are (convex hull works)
coords(p3 p, p3 q, p3 r, p3 s) :
    o(p), dx(q - p), dy(r - p), dz(s - p) {}
// 2D position vector of point p in this coordinate system
// centered at o
// p must be on the plane
PT pos2d(p3 p) {
    return {(p - o) | dx, (p - o) | dy};
}
// returns the 3D position vector of point p in this new
// coordinate system
// p can be outside the plane
p3 pos3d(p3 p) {
    return {(p - o) | dx, (p - o) | dy, (p - o) | dz};
}
// given 2D position vector p centered at o, return the
// original 3D position vector
p3 pos3d(PT p) {
    return o + dx * p.x + dy * p.y;
}
};

struct line3d {
    // d is the direction vector of the line
    p3 d, o; // p = o + k * d (k is a real parameter)
    line3d() {}
    // From two points P, Q
    line3d(p3 p, p3 q) : d(q - p), o(p) {}
    // From two planes p1, p2
    // assuming they are not parallel
    line3d(plane p1, plane p2) {
        d = p1.n * p2.n;
        o = (p2.n * p1.d - p1.n * p2.d) * d / sq(d);
        // o is actually the closest point on the line to the
        // origin
    }
    double dist2(p3 p) { return sq(d * (p - o)) / sq(d); }
    double dist(p3 p) { return sqrt(dist2(p)); }
    // compare points by their projection on the line
    // so you can sort points on the line using this
    bool cmp_proj(p3 p, p3 q) { return (d | p) < (d | q); }
    // orthogonal projection of point p onto line
    p3 proj(p3 p) { return o + d * (d | (p - o)) / sq(d); }
    // orthogonal reflection of point p onto line
    p3 refl(p3 p) { return proj(p) * 2 - p; }
    // returns the intersection point of the line with plane p
    // assuming plane and line are not parallel
    p3 inter(plane p) {
        // assert((d | p.n) != 0); // no intersection if
        // parallel
        return o - d * p.side(o) / (d | p.n);
    }
}
};

```

```

// smallest distance between two lines
double dist(line3d l1, line3d l2) {
    p3 n = l1.d * l2.d;
    if (n == zero) return l1.dist(l2.o); // parallel
    return fabs((l2.o - l1.o) | n) / abs(n);
}
// closest point from line l2 to line l1
p3 closest_on_l1(line3d l1, line3d l2) {
    p3 n2 = l2.d * (l1.d * l2.d);
    return l1.o + l1.d * ((l2.o - l1.o) | n2) / (l1.d | n2);
}
// small angle between direction vectors of two lines
double get_angle(line3d l1, line3d l2) {
    return small_angle(l1.d, l2.d);
}
bool is_parallel(line3d l1, line3d l2) {
    return l1.d * l2.d == zero;
}
bool is_perpendicular(line3d l1, line3d l2) {
    return sign((l1.d | l2.d)) == 0;
}
// small angle between normal vectors of two planes
double get_angle(plane p1, plane p2) {
    return small_angle(p1.n, p2.n);
}
bool is_parallel(plane p1, plane p2) {
    return p1.n * p2.n == zero;
}
bool is_perpendicular(plane p1, plane p2) {
    return sign((p1.n | p2.n)) == 0;
}
double get_angle(plane p, line3d l) {
    return PI / 2 - small_angle(p.n, l.d);
}
bool is_parallel(plane p, line3d l) {
    return sign((p.n | l.d)) == 0;
}
bool is_perpendicular(plane p, line3d l) {
    return p.n * l.d == zero;
}
// returns the line perpendicular to plane p and passing
// through point o
line3d perp_through(plane p, p3 o) {return line3d(o, o + p.n);}
// returns the plane perpendicular to line l and passing
// through point o
plane perp_through(line3d l, p3 o) {return plane(l.d, o);}

// returns two points on intesection line of two planes formed
// by points
// a1, b1, c1 and a2, b2, c2 respectively
pair<p3, p3> plane_plane_intersection(p3 a1, p3 b1, p3 c1, p3
    a2, p3 b2, p3 c2) {
    p3 n1 = (b1 - a1) * (c1 - a1);
    p3 n2 = (b2 - a2) * (c2 - a2);
    double d1 = n1 | a1, d2 = n2 | a2;
    p3 d = n1 * n2;
    if (d == zero) return make_pair(zero, zero);
    p3 o = (n2 * d1 - n1 * d2) * d / (d | d);
    return make_pair(o, o + d);
}
// returns center of circle passing through three
// non-collinear and co-planer points a, b and c
p3 circle_center(p3 a, p3 b, p3 c) {
    p3 v1 = b - a, v2 = c - a;
    double v1v1 = v1 | v1, v2v2 = v2 | v2, v1v2 = v1 | v2;
    double base = 0.5 / (v1v1 * v2v2 - v1v2 * v1v2);
    double k1 = base * v2v2 * (v1v1 - v1v2);
    double k2 = base * v1v1 * (v2v2 - v1v2);
    return a + v1 * k1 + v2 * k2;
}

```

```

}
// segment ab to point c
double distance_from_segment_to_point(p3 a, p3 b, p3 c) {
    if (sign(dot(b - a, c - a)) < 0) return dist(a, c);
    if (sign(dot(a - b, c - b)) < 0) return dist(b, c);
    return fabs(cross((b - a).unit(), c - a).abs());
}

double distance_from_triangle_to_point(p3 a, p3 b, p3 c, p3 d)
{
    plane P(a, b, c);
    p3 proj = P.proj(d);
    double dis = min(distance_from_segment_to_point(a, b, d),
        min(distance_from_segment_to_point(b, c, d),
            distance_from_segment_to_point(c, a, d)));
    int o = sign(orient_by_normal(a, b, proj, P.n));
    int inside = o == sign(orient_by_normal(b, c, proj, P.n));
    inside &= o == sign(orient_by_normal(c, a, proj, P.n));
    if (inside) return (d - proj).abs();
    return dis;
}

double distance_from_triangle_to_segment(p3 a, p3 b, p3 c, p3 d
, p3 e) {
    double l = 0.0, r = 1.0;
    int cnt = 100;
    double ret = inf;
    while (cnt--) {
        double mid1 = l + (r - l) / 3.0, mid2 = r - (r - l) /
            3.0;
        double x = distance_from_triangle_to_point(a, b, c, d +
            (e - d) * mid1);
        double y = distance_from_triangle_to_point(a, b, c, d +
            (e - d) * mid2);
        if (x < y) {
            r = mid2;
            ret = x;
        }
        else {
            ret = y;
            l = mid1;
        }
    }
    return ret;
}

// triangles are solid
double distance_from_triangle_to_triangle(p3 a, p3 b, p3 c, p3
d, p3 e, p3 f) {
    double ret = inf;
    ret = min(ret, distance_from_triangle_to_segment(a, b, c, d
, e));
    ret = min(ret, distance_from_triangle_to_segment(a, b, c, e
, f));
    ret = min(ret, distance_from_triangle_to_segment(a, b, c, f
, d));
    ret = min(ret, distance_from_triangle_to_segment(d, e, f, a
, b));
    ret = min(ret, distance_from_triangle_to_segment(d, e, f, b
, c));
    ret = min(ret, distance_from_triangle_to_segment(d, e, f, c
, a));
    return ret;
}

bool operator < (p3 p, p3 q) {
    return tie(p.x, p.y, p.z) < tie(q.x, q.y, q.z);
}

struct edge {
    int v;
    bool same; // is the common edge between two faces in the
        same order?
};

```

```

};
// Given a series of faces (lists of points) of a polyhedron,
    reverse some of them
// so that their orientations are consistent (all area vectors
    of the faces either pointing outwards or inwards)
// just compute the area vector of one face to see if it's
    pointing outwards or inwards
vector<vector<p3>> reorient(vector<vector<p3>> fs) {
    int n = fs.size();
    // Find the common edges and create the resulting graph
    vector<vector<edge>> g(n);
    map<pair<p3,p3>, int> es;
    for (int u = 0; u < n; u++) {
        for (int i = 0, m = fs[u].size(); i < m; i++) {
            p3 a = fs[u][i], b = fs[u][(i + 1) % m];
            // Let's look at edge a-b
            if (es.count({a, b})) { // seen in same order
                int v = es[{a, b}];
                g[u].push_back({v, true});
                g[v].push_back({u, true});
            }
            else if (es.count({b, a})) { // seen in different
                order
                int v = es[{b, a}];
                g[u].push_back({v, false});
                g[v].push_back({u, false});
            }
            else { // not seen yet
                es[{a, b}] = u;
            }
        }
    }

    // Perform BFS to find which faces should be flipped
    vector<bool> vis(n, false), flip(n);
    flip[0] = false;
    queue<int> q;
    q.push(0);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (edge e : g[u]) {
            if (!vis[e.v]) {
                vis[e.v] = true;
                // If the edge was in the same
                // order,
                // exactly one of the two
                // should be flipped
                flip[e.v] = (flip[u] ^ e.same);
                q.push(e.v);
            }
        }
    }

    // Actually perform the flips
    for (int u = 0; u < n; u++)
        if (flip[u]) {
            reverse(fs[u].begin(), fs[u].end());
        }
    return fs;
}

// O(n^2), O(n) faces in the hull
struct CH3D {
    struct face {
        int a, b, c; // the number of three points on one face
            of the convex hull
        bool ok; // whether the face belongs to the face on
            the final convex hull
    };
    int n; // initial vertex number
};

```

```

vector<p3> P;
int num; // convex hull surface triangle number
vector<face> F; // convex surface triangles
vector<vector<int>> g;

void init(vector<p3> p) {
    P = p;
    n = p.size();
    F.resize(8 * n + 1);
    g.resize(n + 1, vector<int> (n + 1));
}

double len(p3 a) {
    return sqrt(a | a);
}

p3 cross(const p3 &a, const p3 &b, const p3 &c) {
    return (b - a) * (c - a);
}

double area(p3 a, p3 b, p3 c) {
    return len((b - a) * (c - a));
}

double volume(p3 a, p3 b, p3 c, p3 d) {
    return (b - a) * (c - a) | (d - a);
}

// positive: p3 in the same direction
double dblcmp(p3 &p, face &f) {
    p3 m = P[f.b] - P[f.a];
    p3 n = P[f.c] - P[f.a];
    p3 t = p - P[f.a];
    return (m * n) | t;
}

void deal(int p, int a, int b) {
    int f = g[a][b]; // search for another plane adjacent
        to the edge
    face add;
    if (F[f].ok) {
        if (dblcmp(P[p], F[f]) > eps) dfs(p, f);
        else {
            add.a = b;
            add.b = a;
            add.c = p; // pay attention to the order here,
                to be right-handed
            add.ok = true;
            g[p][b] = g[a][p] = g[b][a] = num;
            F[num++] = add;
        }
    }
}

// recursively search all faces that should be removed from
    the convex hull
void dfs(int p, int now) {
    F[now].ok = 0;
    deal(p, F[now].b, F[now].a);
    deal(p, F[now].c, F[now].b);
    deal(p, F[now].a, F[now].c);
}

bool same(int s, int t) {
    p3 &a = P[F[s].a];
    p3 &b = P[F[s].b];
    p3 &c = P[F[s].c];
    return fabs(volume(a, b, c, P[F[t].a])) < eps &&
        fabs(volume(a, b, c, P[F[t].b])) < eps &&
        fabs(volume(a, b, c, P[F[t].c])) < eps;
}

// building a 3D convex hull
void create_hull() {
    int i, j, tmp;
    face add;
    num = 0;
};

```

```

    if (n < 4) return;

    // ensure that the first four points are not coplanar
    bool flag = true;
    for (i = 1; i < n; i++) {
        if (len(P[0] - P[i]) > eps) {
            swap(P[1], P[i]);
            flag = false;
            break;
        }
    }
    if (flag) return;
    flag = true;
    // make the first three points not collinear
    for (i = 2; i < n; i++) {
        if (len((P[0] - P[1]) * (P[1] - P[i])) > eps) {
            swap(P[2], P[i]);
            flag = false;
            break;
        }
    }
    if (flag) return;
    flag = true;
    // make the first four points not coplanar
    for (int i = 3; i < n; i++) {
        if (fabs((P[0] - P[1]) * (P[1] - P[2]) | (P[0] - P[
            i])) > eps) {
            swap(P[3], P[i]);
            flag = false;
            break;
        }
    }
    if (flag) return;

    for (i = 0; i < 4; i++) {
        add.a = (i + 1) % 4;
        add.b = (i + 2) % 4;
        add.c = (i + 3) % 4;
        add.ok = true;
        if (dblcmp(P[i], add) > 0) swap(add.b, add.c);
        g[add.a][add.b] = g[add.b][add.c] = g[add.c][add.a]
            = num;
        F[num++] = add;
    }
    for (i = 4; i < n; i++) {
        for (j = 0; j < num; j++) {
            if (F[j].ok && dblcmp(P[i], F[j]) > eps) {
                dfs(i, j);
                break;
            }
        }
    }
    tmp = num;
    for (i = num = 0; i < tmp; i++)
        if (F[i].ok) F[num++] = F[i];
}

double surface_area() {
    double res = 0;
    if (n == 3) {
        p3 p = cross(P[0], P[1], P[2]);
        res = len(p) / 2.0;
        return res;
    }
    for (int i = 0; i < num; i++) {
        res += area(P[F[i].a], P[F[i].b], P[F[i].c]);
    }
    return res / 2.0;
}

```

```

double volume() {
    double res = 0;
    p3 tmp(0, 0, 0);
    for (int i = 0; i < num; i++) {
        res += volume(tmp, P[F[i].a], P[F[i].b], P[F[i].c])
            ;
    }
    return fabs(res / 6.0);
}

int number_of_triangles() { // number of surface triangles
    return num;
}

int number_of_polygons() { // number of surface polygons
    int i, j, res, flag;
    for (i = res = 0; i < num; i++) {
        flag = 1;
        for (j = 0; j < i; j++) {
            if (same(i, j)) {
                flag = 0;
                break;
            }
        }
        res += flag;
    }
    return res;
}

p3 centroid() { // center of gravity
    p3 ans(0, 0, 0), o(0, 0, 0);
    double all = 0;
    for (int i = 0; i < num; i++) {
        double vol = volume(o, P[F[i].a], P[F[i].b], P[F[i]
            ].c));
        ans = ans + (o + P[F[i].a] + P[F[i].b] + P[F[i].c])
            / 4.0 * vol;
        all += vol;
    }
    ans = ans / all;
    return ans;
}

double point_to_face_distance(p3 p, int i) {
    return fabs(volume(P[F[i].a], P[F[i].b], P[F[i].c], p)
        / len((P[F[i].b] - P[F[i].a]) * (P[F[i].c] - P[F[i]
            ].a))));
}

};

// https://desktop.arcgis.com/en/arcmap/10.7/map/projections/
// geographic-coordinate-system.htm
// given the radius of the sphere, latitude and longitude of a
// point in degrees
// return the 3D coordinates of the point on the sphere
// assuming the sphere is centered at the origin
p3 get_sphere(double r, double lat, double lon) {
    lat *= PI / 180, lon *= PI / 180;
    return {r * cos(lat) * cos(lon), r * cos(lat) * sin(lon), r
        * sin(lat)};
}

int sphere_line_intersection(p3 o, double r, line3d l, pair<p3,
    p3> &out) {
    double h2 = r * r - l.dist2(o);
    if (h2 < 0) return 0; // the line doesn't touch the sphere
    p3 p = l.proj(o);
    p3 h = l.d * sqrt(h2) / abs(l.d); // vector parallel to l, of
        length h
    out = {p - h, p + h};
    return 1 + (h2 > 0);
}

// The shortest distance between two points A and B on a sphere
// (O, r) is

```

```

// given by travelling along plane OAB and on the surface of
// the sphere. It is called the great-circle distance
// if a and b are outside the sphere, then it will give the
// distance between their projections on the sphere
double great_circle_dist(p3 o, double r, p3 a, p3 b) {
    // s = r * theta
    return r * get_angle(a - o, b - o);
}

// Assume that the sphere is centered at the origin
// We will call a segment [AB] valid if A and B are not
// opposite each other on the sphere
bool validSegment(p3 a, p3 b) {
    return a * b != zero || (a | b) > 0;
}

bool proper_intersection(p3 a, p3 b, p3 c, p3 d, p3 &out) {
    p3 ab = a * b, cd = c * d; // normals of planes OAB and OCD
    int oa = sign(cd | a),
        ob = sign(cd | b),
        oc = sign(ab | c),
        od = sign(ab | d);
    out = ab * cd * od; // four multiplications => careful with
        overflow!
    return (oa != ob && oc != od && oa != oc);
}

// Assume that the sphere is centered at the origin
bool point_on_sphere_segment(p3 a, p3 b, p3 p) {
    p3 n = a * b;
    if (n == zero)
        return a * p == zero && (a | p) > 0;
    return (n | p) == 0 && (n | a * p) >= 0 && (n | b * p) <=
        0;
}

struct DirectionSet : vector<p3> {
    using vector::vector; // import constructors
    void insert(p3 p) {
        for (p3 q : *this) if (p * q == zero) return;
        push_back(p);
    }
};

// Assume that the sphere is centered at the origin
// it returns the direction vectors of the intersection points
// to get the actual points, scale the direction vectors to the
// radius of the sphere
DirectionSet segment_intersection_on_sphere(p3 a, p3 b,
    p3 c, p3 d) {
    assert(validSegment(a, b) && validSegment(c, d));
    p3 out;
    if (proper_intersection(a, b, c, d, out)) return {out};
    DirectionSet s;
    if (point_on_sphere_segment(c, d, a)) s.insert(a);
    if (point_on_sphere_segment(c, d, b)) s.insert(b);
    if (point_on_sphere_segment(a, b, c)) s.insert(c);
    if (point_on_sphere_segment(a, b, d)) s.insert(d);
    return s;
}

// small angle between spherical segments ab and ac
// assume that the sphere is centered at the origin
// all points a, b, c are on the sphere
double angle_on_sphere(p3 a, p3 b, p3 c) {
    return get_angle(a * b, a * c);
}

// oriented angle between spherical segments ab and ac

```

```
// that is how much we rotate counterclockwise to get from ab
// to ac
// assume that the sphere is centered at the origin
// all points a, b, c are on the sphere
double oriented_angle_on_sphere(p3 a, p3 b, p3 c) {
    if ((a * b | c) >= 0) return angle_on_sphere(a, b, c);
    else return 2 * PI - angle_on_sphere(a, b, c);
}

// Assume that the sphere is centered at the origin
// the polygon is simple and given in counterclockwise order
// for each consecutive pair of points, the counterclockwise
// left
// part of the segment is considered to be inside the surface
// area that the polygon encloses
// if the polygon is outside the sphere, the projection of the
// polygon on the sphere will be considered
double area_on_the_surface_of_the_sphere(double r, vector<p3> p
) {
    int n = p.size();
    double sum = -(n - 2) * PI;
    for (int i = 0; i < n; i++) {
        sum += oriented_angle_on_sphere(p[(i + 1) % n], p[(i +
        2) % n], p[i]);
    }
    return r * r * sum;
}

// Assume that O is the origin
// it returns 0 if O is outside the polyhedron
// 1 if O is inside the polyhedron, and the vector areas of the
// faces are oriented towards the outside;
// -1 if O is inside the polyhedron, and the vector areas of
// the faces are oriented towards the inside.
int winding_number_3D(vector<vector<p3>> fs) {
    double sum = 0;
    for (vector<p3> f : fs) {
        sum += remainder(area_on_the_surface_of_the_sphere(1, f
        ), 4 * PI);
    }
    return round(sum / (4 * PI));
}

struct sphere {
    p3 c;
    double r;
    sphere() {}
    sphere(p3 c, double r) : c(c), r(r) {}
};

// spherical cap is a portion of a sphere cut off by a plane
// https://en.wikipedia.org/wiki/Spherical_cap
struct spherical_cap {
    p3 c;
    double r;
    spherical_cap() {}
    spherical_cap(p3 c, double r) : c(c), r(r) {}

    // angle th is the polar angle between the rays from the
    // center of the sphere to one edge of the cap
    // and orthogonal line from the center of the sphere to the
    // plane of the cap

    // height of the cap (just like real world cap)
    double height(double th) {
        return r * (1 - cos(th));
    }
    // radius of the base of the cap
    double base_radius(double th) {
```

```
        return r * sin(th);
    }
    // volume of the cap
    double volume(double th) {
        double h = height(th);
        return PI * h * h * (3 * r - h) / 3.0;
    }
    // surface area of the cap
    double surface_area(double th) {
        double h = height(th);
        return 2 * PI * r * h;
    }
};

// returns the sphere passing through four points
sphere circumscribed_sphere(p3 a, p3 b, p3 c, p3 d) {
    assert( sign(plane(a, b, c).side(d)) != 0);

    plane u = plane(a - b, (a + b) / 2);
    plane v = plane(b - c, (b + c) / 2);
    plane w = plane(c - d, (c + d) / 2);

    assert(!is_parallel(u, v));
    assert(!is_parallel(v, w));
    line3d l1(u, v), l2(v, w);
    assert( sign(dist(l1, l2)) == 0);

    p3 C = closest_on_l1(l1, l2);
    return sphere(C, abs(C - a));
}

// https://mathworld.wolfram.com/Sphere-SphereIntersection.html
// it won't work if one sphere is totally inside the other
// sphere
// handle that case separately
// returns the surface area and volume of the intersection
pair<double, double> sphere_sphere_intersection(sphere s1,
    sphere s2) {
    double d = abs(s1.c - s2.c);
    if(sign(d - s1.r - s2.r) >= 0) return {0, 0}; // not
    intersecting
    // only the distance matters, so we will now consider the
    // centers
    // of the big sphere to be (0, 0, 0) and (d, 0, 0) for the
    // small sphere
    // we can transform the results back to w.r.t the real
    // centers if we want

    double R = max(s1.r, s2.r);
    double r = min(s1.r, s2.r);
    double y = R + r - d;
    double x = (R * R - r * r + d * d) / (2 * d);
    // the intersecting plane is parallel to the yz plane
    // with the above x value as its x coordinate
    double w = d * d - r * r + R * R;
    double a = sqrt(4 * d * d * R * R - w * w) / (2.0 * d);
    // a is the radius of the intersecting circle on the
    // intersecting plane
    // with center (x, 0)
    double h1 = R - x;
    double h2 = y - h1;
    // h1 is the height of the intersecting spherical cap of
    // the big sphere
    // h2 is for the small sphere

    // total volume of the whole intersection = sum of the
    // volumes of the spherical caps
    double volume = PI * h1 * h1 * (3 * R - h1) / 3.0 + PI
        * h2 * h2 * (3 * r - h2) / 3.0;
```

```
// total surface area of the intersecting spherical caps
double surface_area = 2 * PI * R * h1 + 2 * PI * r * h2;
return make_pair(surface_area, volume);
}

sphere smallest_enclosing_sphere(vector<p3> p) {
    int n = p.size();
    p3 c(0, 0, 0);
    for(int i = 0; i < n; i++) c = c + p[i];
    c = c / n;

    double ratio = 0.1;
    int pos = 0;
    int it = 100000;
    while (it--) {
        pos = 0;
        for (int i = 1; i < n; i++) {
            if(sq(c - p[i]) > sq(c - p[pos])) pos = i;
        }
        c = c + (p[pos] - c) * ratio;
        ratio *= 0.998;
    }
    return sphere(c, abs(c - p[pos]));
}

// it returns the angle of the spherical cap that is formed by
// the intersection of all tangents
double tangent_from_point_to_sphere(p3 p, sphere s) {
    double d = abs(p - s.c);
    if (sign(d - s.r) < 0) return -1; // inside the sphere, so
    no tangent
    if (sign(d - s.r) == 0) return -2; // on the sphere, handle
    separately
    double tangent_length = sqrt(d * d - s.r * s.r);
    double th = acos(s.r / d);
    return th;
}

struct pyramid {
    int n; // number of sides of the pyramid
    double l; // length of each side
    double ang;
    pyramid(int _n, double _l) {
        n = _n;
        l = _l;
        ang = PI / n;
    }
    double base_area() {
        return l * l * n / (4 * tan(ang));
    }
    double height() {
        return l * sqrt(1 - 1 / (4 * sin(ang) * sin(ang)))
        ;
    }
    double volume() {
        return base_area() * height() / 3;
    }
};

struct cylinder {
    double r, h; // radius and height
    cylinder(double _r, double _h) {
        r = _r;
        h = _h;
    }
    double volume() {
        return PI * r * r * h;
    }
    double surface_area() {
```

```
        return 2 * PI * r * h + 2 * PI * r * r;
    }
};

struct cone {
    double r, h; // radius and height
    cone(double _r, double _h) {
        r = _r;
        h = _h;
    }
    double volume() {
        return PI * r * r * h / 3.0;
    }
    double surface_area() {
        return PI * r * (r + sqrt(h * h + r * r));
    }
};

int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    return 0;
}
```

Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

```
vi pi(const string& s) {
    vi p(sz(s));
    rep(i,1,sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

```
vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i,sz(p)-sz(s),sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

```
vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i,1,sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

```
array<vi, 2> manacher(const string& s) {
    int n = sz(s);
    array<vi,2> p = {vi(n+1), vi(n)};
    rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
        int t = r-i+!z;
        if (i<r) p[z][i] = min(t, p[z][l+t]);
        int L = i-p[z][i], R = i+p[z][i]-!z;
        while (L>=1 && R+1<n && s[L-1] == s[R+1])
            p[z][i]++, L--, R++;
        if (R>r) l=L, r=R;
    }
    return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());

Time: $\mathcal{O}(N)$

```
int minRotation(string s) {
    int a=0, N=sz(s); s += s;
    rep(b,0,N) rep(k,0,N) {
        if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
        if (s[a+k] > s[b+k]) {a = b; break;}
    }
    return a;
}
```

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size n + 1, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars.

Time: $\mathcal{O}(n \log n)$

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256) { // or vector<int>
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                s[i + k] == s[j + k]; k++);
    }
};
```

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r) into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).

Time: $\mathcal{O}(26N)$

```
struct SuffixTree {
    enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
    string a; // v = cur node, q = cur position
    int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;

    void ukkadd(int i, int c) { suff:
        if (r[v]<=q) {
            if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
                p[m++]=v; v=s[v]; q=r[v]; goto suff; }
            v=t[v][c]; q=l[v];
        }
        if (q==-1 || c==toi(a[q])) q++; else {
            l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
            p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
            l[v]=q; p[v]=m; t[p[m]][toi(a[l[m]])]=m;
            v=s[p[m]]; q=l[m];
            while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
            if (q==r[m]) s[m]=v; else s[m]=m+2;
            q=r[v]-(q-r[m]); m+=2; goto suff;
        }
    }

    SuffixTree(string a) : a(a) {
        fill(r,r+N,sz(a));
        memset(s, 0, sizeof s);
        memset(t, -1, sizeof t);
        fill(t[1],t[1]+ALPHA,0);
        s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
        rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
    }
}
```

```
// example: find longest common substring (uses ALPHA = 28)
pii best;
int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
        mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)
        best = max(best, {len, r[node] - len});
    return mask;
}
static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
}
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.

```
const int MOD1 = 1000000007;
const int MOD2 = 1000000009;
const int BASE = 911382323;

struct DoubleHash {
    vector<int> pow1, pow2, hash1, hash2;
```

```
DoubleHash(const string &s) {
    int n = s.size();
    pow1.resize(n + 1);
    pow2.resize(n + 1);
    hash1.resize(n + 1);
    hash2.resize(n + 1);

    pow1[0] = pow2[0] = 1;
    hash1[0] = hash2[0] = 0;

    for (int i = 0; i < n; i++) {
        pow1[i + 1] = (1LL * pow1[i] * BASE) % MOD1;
        pow2[i + 1] = (1LL * pow2[i] * BASE) % MOD2;

        hash1[i + 1] = ( (1LL * hash1[i] * BASE) + (s[i] - 'a' + 1) ) % MOD1;
        hash2[i + 1] = ( (1LL * hash2[i] * BASE) + (s[i] - 'a' + 1) ) % MOD2;
    }
}

pair<int,int> get_hash(int l, int r) {
    int h1 = (hash1[r+1] - 1LL * hash1[l] * pow1[r-l+1] % MOD1 + MOD1) % MOD1;
    int h2 = (hash2[r+1] - 1LL * hash2[l] * pow2[r-l+1] % MOD2 + MOD2) % MOD2;
    return {h1, h2};
}

};

signed main() {
    ios::sync_with_stdio(false); cin.tie(0);

    string s = "amartya";
    MultiHash hasher;
    hasher.init(s);

    auto h1 = hasher.get_hash(0, 2); // "ama"
    auto h2 = hasher.get_hash(3, 6); // "rtya"
    auto full = hasher.get_hash(0, 6); // "amartya"

    auto combined = hasher.concat(h1, h2, 4);

    cout << (hasher.compare(combined, full) ? "Correct concat "
        : "Something's off ") << "\n";
}
```

AhoCorasick.h
Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(–, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.
Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.
f35677, 66 lines

```
struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vi backp;
    void insert(string& s, int j) {
```

```
assert(!s.empty());
    int n = 0;
    for (char c : s) {
        int& m = N[n].next[c - first];
        if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
        else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
}

AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i, 0, sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
        int n = q.front(), prev = N[n].back;
        rep(i, 0, alpha) {
            int &ed = N[n].next[i], y = N[prev].next[i];
            if (ed == -1) ed = y;
            else {
                N[ed].back = y;
                (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                    = N[y].end;
                N[ed].nmatches += N[y].nmatches;
                q.push(ed);
            }
        }
    }
}

vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}

vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
        int ind = r[i];
        while (ind != -1) {
            res[i - sz(pat[ind]) + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};
```

Various (10)

10.1 Debugging tricks

- signal(SIGSEGV, [](int) { _Exit(0); });
converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). _GLIBCXX_DEBUG failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).

- feenableexcept(29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.2 Optimization tricks

__builtin_ia32_ldmxcsr(40896); disables denormals (which make floats 20x slower near their minimum value).

10.2.1 Bit hacks

- x & -x is the least bit in x.
- for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- c = x&-x, r = x+c; (((r^x) >> 2)/c) | r is the next number after x with the same number of bits set.
- rep(b, 0, K) rep(i, 0, (1 << K))
if (i & 1 << b) D[i] += D[i^(1 << b)];
computes all sums of subsets.

10.2.2 Pragmas

- #pragma GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- #pragma GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.
- #pragma GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

```
int128.h
Description: int128 for 128 bit integer
Usage: int 128 doesnot support cout cin
Time:  $\mathcal{O}()$ 
a5d97e, 15 lines

__int128 read() {
    __int128 x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') {
        if (ch == '-') f = -1;
        ch = getchar();
    } while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0'; ch = getchar();
    } return x * f;
}

void print(__int128 x) {
    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) print(x / 10);
    putchar(x % 10 + '0');
}

bool cmp(__int128 x, __int128 y) { return x > y; }
__int128 abs_int128(__int128 x) {return x < 0 ? -x : x;}
```

Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients
Pigeon-hole principle
Inclusion/exclusion
Catalan number
Pick's theorem
Number theory
Integer parts
Divisibility
Euclidean algorithm
Modular arithmetic
* Modular multiplication
* Modular inverses
* Modular exponentiation by squaring
Chinese remainder theorem
Fermat's little theorem
Euler's theorem
Phi function
Frobenius number
Quadratic reciprocity
Pollard-Rho
Miller-Rabin
Hensel lifting
Vieta root jumping
Game theory
Combinatorial games
Game trees
Mini-max
Nim
Games on graphs
Games on graphs with loops
Grundy numbers
Bipartite games without repetition
General games without repetition
Alpha-beta pruning
Probability theory
Optimization
Binary search
Ternary search
Unimodality and convex functions
Binary search on derivative
Numerical methods
Numeric integration
Newton's method
Root-finding with binary/ternary search
Golden section search
Matrices
Gaussian elimination
Exponentiation by squaring
Sorting
Radix sort
Geometry
Coordinates and vectors
* Cross product
* Scalar product
Convex hull
Polygon cut
Closest pair
Coordinate-compression
Quadtrees
KD-trees
All segment-segment intersection
Sweeping
Discretization (convert to events and sweep)
Angle sweeping
Line sweeping
Discrete second derivatives
Strings
Longest common substring
Palindrome subsequences

Knuth-Morris-Pratt
Tries
Rolling polynomial hashes
Suffix array
Suffix tree
Aho-Corasick
Manacher's algorithm
Letter position lists
Combinatorial search
Meet in the middle
Brute-force with pruning
Best-first (A*)
Bidirectional search
Iterative deepening DFS / A*
Data structures
LCA (2^k-jumps in trees in general)
Pull/push-technique on trees
Heavy-light decomposition
Centroid decomposition
Lazy propagation
Self-balancing trees
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
Monotone queues / monotone stacks / sliding queues
Sliding queue using 2 stacks
Persistent segment tree